

Softwareprojekt-Engineering II

Zusammenfassung der Vorlesung

Lukas Hanser

Software Engineering
FH Oberösterreich – Campus Hagenberg
Sommersemester 2026

Inhaltsverzeichnis

1	Anforderungsanalyse	1
1.1	Ziele	1
1.1.1	Problemzerlegung (Analyse)	1
1.1.2	Definition im Bereich der Softwareentwicklung	1
1.2	Aufgaben	2
1.2.1	Tätigkeiten	2
1.2.2	Eigenschaften einer guten Anforderungsanalyse	2
1.2.3	Schwierigkeiten	3
1.3	Techniken	4
1.3.1	Komplexitätstechniken	4
1.3.2	Zerlegungstechniken	5
1.3.3	Aktuelle Trends bei der Zerlegung	8
1.3.4	Erstellen von Prototypen	9
1.4	Werkzeuge	11
1.4.1	UML	11
1.4.2	Prototyping	12
1.5	Ergebnisse	13
1.5.1	Analysebericht	13
1.5.2	Lastenheft	13
1.5.3	Machbarkeitsstudie	14
2	Planung	15
2.1	Ziel	15
2.2	Aufgaben	16
2.2.1	Organisationsplanung	16
2.2.2	Ziel- und Aufgabenplanung	16
2.2.3	Zeitplanung	16
2.2.4	Ablauf- und Terminplanung	17
2.2.5	Ressourcenplanung	17
2.2.6	Kosten- und Finanzierungsplanung	17
2.2.7	Begleitende und vorsorgende Maßnahmen	17
2.3	Iterativ-inkrementelle Planung beim agilen Vorgehen	18
2.3.1	Struktur moderner Werkzeuge	18
2.3.2	Anforderungen und Aufgaben	18
2.3.3	Planung Projekt	19
2.3.4	Planung Anforderungen	19
2.3.5	Planung Aufgaben	19
2.3.6	Planung Iterationen	20
2.3.7	Kombiniertes Push-Pull-Prinzip	20
2.4	Techniken	21
2.4.1	Aufwandsschätzung	21
2.4.2	Netzplantechnik	21
2.4.3	Balkenplantechnik	22

2.4.4	Laufendes Schätzen des Restaufwands	22
2.5	Werkzeuge	23
2.6	Ergebnisse	23
3	Risikobehandlung	24
3.1	Definition	24
3.2	Ziele	24
3.2.1	Notwendigkeit von Risiken	24
3.2.2	Bestimmung des Risikofaktors	24
3.3	Aufgaben	25
3.3.1	Risiko-Identifikation	25
3.3.2	Risiko-Analyse	26
3.3.3	Risiko-Evaluierung	26
3.3.4	Risiko-Vorsorgeplan	26
3.3.5	Risiko-Überwachung	26
3.3.6	Risiko-Überwindung	26
3.4	Techniken	27
3.4.1	Abschätzen des Risikoverhaltens	27
3.4.2	Strategie der Risikomitigation	27
3.4.3	Mitigation der wichtigsten Risikoelemente	27
3.4.4	Verwenden von Standardprodukten	28
3.4.5	Fehlerbaumanalyse (FTA)	28
3.4.6	Fehler-Möglichkeiten und Einfluss-Analyse (FMEA)	29
3.4.7	SWOT-Analyse	30
3.5	Werkzeuge	30
3.5.1	Risiko-Fieberkurve	30
3.5.2	Risiko-Hitzekarte	30
3.6	Ergebnisse	31
3.6.1	Top-N-Liste	31
3.6.2	Risiko-Vorsorgeplan	31
3.7	Risikomanagement	32
3.7.1	Motivation	32
3.7.2	Basis Risikopolitik	32
3.7.3	Prozess	32
3.7.4	Stellen	33
4	Design	34
4.1	Ziel	34
4.2	Aufgaben	34
4.2.1	Designmodell	34
4.2.2	Gliederung	35
4.2.3	Modellierung der Architektur	35
4.2.4	Modellierung der Schnittstellen	36
4.2.5	Modellierung der Datenhaltung	36
4.2.6	Modellierung der Struktur	36

4.2.7	Modellierung des Verhaltens	37
4.3	Techniken	38
4.3.1	Erweitern der Analysemodelle	38
4.3.2	Erstellen von Prototypen	39
4.3.3	Modellgesteuertes Entwickeln	39
4.3.4	Endbenutzer-Entwicklung	40
4.3.5	Wiederverwendung	40
4.3.6	Empfehlungen für ein gutes Design	41
4.4	Werkzeuge	42
4.4.1	Computergestützte Designwerkzeuge	42
4.4.2	Unified Modeling Language (UML)	42
4.4.3	Prototyping	43
4.5	Ergebnisse	44
4.5.1	Designbericht	44
4.5.2	Pflichtenheft	44
4.5.3	Benutzerdokumentation	45
5	Implementierung - Codieren	46
5.1	Ziel	46
5.1.1	Aufgaben	46
5.1.2	Geschichte der Programmierung	46
5.2	Techniken	47
5.2.1	Transformieren des Designs	47
5.2.2	Einfügen logischer Bedingungen	47
5.2.3	Instrumentieren des Codes	47
5.2.4	Kontinuierliche Integration	48
5.2.5	Code-Restrukturierung (Refactoring)	48
5.2.6	Erstellen von Testrahmen	49
5.3	Testgesteuerte Entwicklung	50
5.3.1	Idee	50
5.3.2	Ablauf	50
5.3.3	Konsequenz	50
5.4	Beispiel	51
5.4.1	Vorgehen	51
5.4.2	Vorteile und Herausforderungen	52
5.5	Werkzeuge	54
5.6	Ergebnisse	54
5.6.1	Quellcode	54
5.6.2	Systemdokumentation	54
6	Implementierung - Debuggen	55
6.1	Ziel	55
6.2	Aufgaben	55
6.3	Techniken	55
6.3.1	Statische Programmanalyse	56

6.3.2	Dynamische Programmanalyse	57
6.3.3	Effizienz der Techniken beim Finden von Fehlern	57
6.4	Werkzeuge	58
6.4.1	Statische Analysatoren	58
6.4.2	Listengeneratoren	58
6.4.3	Speicherprüfer	58
6.4.4	Testrahmensysteme	58
6.4.5	Debugger	58
6.5	Ergebnisse	59
6.5.1	Lauffähiges Programm	59
6.5.2	Fehlerliste	59
7	Testen	60
7.1	Ziel	60
7.1.1	Ziel aus Stakeholdersicht	60
7.1.2	Rollenverteilung	60
7.2	Aufgaben	61
7.2.1	Bestimmen des Produktstatus	61
7.2.2	Testen der Produktperformannz	62
7.2.3	Vorbereiten der Abnahme	62
7.3	Techniken	63
7.3.1	Testfalldesign	63
7.3.2	Testsuitesdesign	63
7.3.3	Testfallgenerierung	64
7.3.4	Testfallauswahl	64
7.3.5	Testablaufplanung	64
7.3.6	Regressionstesten	65
7.3.7	Testautomatisierung	66
7.4	Verifikation vs. Validierung	67
7.4.1	Verifikation	67
7.4.2	Validierung	67
7.5	Werkzeuge	68
7.6	Ergebnisse	69
7.6.1	Testplan	69
7.6.2	Testsuite	69
7.6.3	Testliste	70
7.6.4	Fehlerbericht	70
7.6.5	Fehlerlogbuch	70
8	Produkteinführung	71
8.1	Ziel	71
8.2	Aufgaben	71
8.2.1	Lieferung	71
8.2.2	Installation und Inbetriebnahme	72
8.2.3	Schulung	72

8.2.4	Abnahme und Abschluss	72
8.2.5	Evaluierung	73
8.2.6	Archivierung	73
8.2.7	Wartung	73
8.2.8	Wartungsalternativen	74
8.3	Techniken	75
8.3.1	Umstellung	75
8.3.2	Probetrieb	75
8.3.3	Abnahmetest	75
8.3.4	Nachkalkulation	76
8.3.5	Kontinuierliche Lieferung/Freigabe	76
8.3.6	Development & Operations (DevOps)	77
8.3.7	Alternative zur Wartung	77
8.3.8	Sanierung	78
8.3.9	Evolution	79
8.4	Werkzeuge	80
8.5	Ergebnisse	80
8.5.1	Betriebsversion	80
8.5.2	Installations- und Inbetriebnahmeanleitung	80
8.5.3	Abnahmeprotokoll	81
8.5.4	Abschlussbericht	81
8.5.5	Projektarchiv	81
9	Tipps für die agile Softwareentwicklung	82
9.1	Denken in Iterationen	82
9.1.1	Konzept	82
9.1.2	Geregelter Prozess	82
9.2	Denken in Zuständen	83
9.3	Aufgabenübersicht (Task Board)	83
9.4	Anforderungen/Aufgaben versus Iterationen	84
9.5	Scrum Task Board	84
9.6	Kanban	85
9.6.1	Abgrenzung	85
9.6.2	Empfehlungen	86
9.6.3	Darstellung am Kanban-Board	86
9.6.4	Kortschritt und Zielerreichung	86
9.6.5	Hauptnutzen	86
9.7	Tipps zum (agilen) Planen	87
9.7.1	Statische Zusammenhänge	87
9.7.2	Dynamische Zusammenhänge	87
9.7.3	Erarbeiten der Ablauflogik	88
9.7.4	Erstellen typischer Anwendungsfälle	88
9.7.5	Interne versus externe Anforderungen/Aufgaben	88
9.7.6	Sonderfall Forschungsaufgabe	89
9.7.7	Umplanen der aktuellen Iteration	89

9.7.8	Vorplanen zukünftiger Iterationen	89
9.8	Tipps zum (agilen) Durchführen	90
9.8.1	Erfassen der Arbeitszeit	90
9.8.2	Schätzen des Restaufwands	90
9.8.3	Ändern des Zustands von Aufgaben (Status)	90
9.9	Tipps zum (agilen Überprüfen)	91
9.9.1	Aufgaben - Fortschritt / Zielerreichung	91
9.9.2	Anforderungen - Fortschritt / Zielerreichung	92
9.9.3	Management-Overhead	93
9.9.4	Auslastung der Mitarbeiter	93
9.9.5	Analyse des Schätzvermögens	94
9.9.6	Potenzial der Prozessverbesserung	94
10	Trends bei moderner (agiler) Softwareentwicklung	95
10.1	Beispiele agiler Vorgehensmodelle	95
10.1.1	Prozessmodell-Topologie	95
10.1.2	Prozess-Skalierung	95
10.1.3	Agile Vorgehensmodelle	96
10.2	Agile Vorgehensmodelle für größere Unternehmen	97
10.2.1	Beobachtungen zu agilen Vorgehensmodellen	97
10.2.2	Beispiel: Scrum wird immer fetter	97
10.2.3	Agile Vorgehen werden breiter einsetzbar	97

1 Anforderungsanalyse

1.1 Ziele

Grundlage

„Man soll jede schwierige Frage in so viele Teilfragen wie möglich zerlegen, damit man jede einzelne lösen kann.“

1.1.1 Problemzerlegung (Analyse)

Aspekt	Funktionale Anforderungen	Nichtfunktionale Anforderungen
Frage	Was soll das System tun?	Wie gut bzw. unter welchen Bedingungen arbeitet das System?
Fokus	Funktionen und Verhalten	Qualität, Einschränkungen, Rahmenbedingungen
Beschreibung	Beschreibt konkrete Features und Aktionen	Beschreibt Qualitätsmerkmale des Systems
Beispiel	Benutzer kann sich anmelden und Daten speichern	Login dauert weniger als 2 Sekunden
Testbarkeit	Direkt über Ein-/Ausgabe testbar	Oft über Messwerte (Performance, Sicherheit, etc.)

Tabelle 1: Vergleich funktionaler und nichtfunktionaler Anforderungen

1.1.2 Definition im Bereich der Softwareentwicklung

Definition: T. DeMarco, 1978

„Studieren eines Problems, noch bevor Aktionen getätigt werden.“

Definition: P.Coad & E. Yourdon, 1990

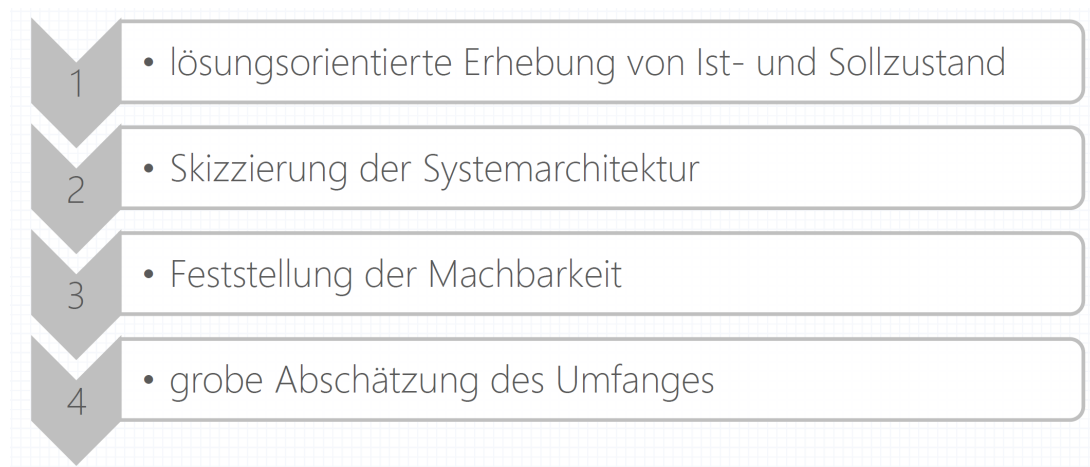
„Studieren eines Problemraumes zur Beschreibung des externen Systemverhaltens.“

Merke

WAS/WIE - Trennung ist entscheidend!

1.2 Aufgaben

1.2.1 Tätigkeiten



Merke

Verifikation und **Validierung** der Anforderungen sind entscheidend!

1.2.2 Eigenschaften einer guten Anforderungsanalyse

- **Abgrenzen des Projektumfangs:** Festlegen, welche Funktionen Teil des Systems sind und welche bewusst ausgeschlossen werden.
- **Modularität beachten:** Zerlegung des Systems in klar getrennte, wiederverwendbare und unabhängig entwickelbare Bausteine.
- **Änderbarkeit sicherstellen:** Gestaltung des Systems so, dass zukünftige Anpassungen mit möglichst geringem Aufwand möglich sind.
- **Schnittstellen definieren:** Festlegung klarer Kommunikationspunkte zwischen Systemteilen und externen Systemen.
- **Rahmenbedingungen (Constraints) festlegen:** Vorgaben und Einschränkungen wie technische, rechtliche oder zeitliche Bedingungen definieren.
- **Realisierbarkeit prüfen:** Bewertung, ob das System unter den gegebenen Ressourcen und Bedingungen umsetzbar ist.

Beispiel: Anforderungsanalyse eines Online-Shops

- **Abgrenzen des Projektumfanges:** Der Online-Shop umfasst nur Produktkatalog, Warenkorb und Bestellprozess. Zahlungsabwicklung über externe Anbieter ist nicht Teil des Systems.
- **Achten auf Modularität:** Das System wird in Module wie Benutzerverwaltung, Produktverwaltung und Bestellabwicklung getrennt.
- **Achten auf Änderbarkeit:** Produktkategorien und Preise sollen leicht anpassbar sein, ohne dass der gesamte Code geändert werden muss.
- **Festlegen der Schnittstellen:** Es werden APIs für Zahlungssysteme (z.B. PayPal) und Versanddienstleister definiert.
- **Festlegen von Constraints:** Das System muss DSGVO-konform sein und Bestellungen innerhalb von 2 Sekunden verarbeiten.
- **Prüfen der Realisierbarkeit:** Es wird geprüft, ob die gewünschte Last von 10.000 gleichzeitigen Nutzern mit der geplanten Infrastruktur erreichbar ist.

1.2.3 Schwierigkeiten

- **Verständnis für das Anwendungsgebiet:** Ein tiefes Verständnis der fachlichen Domäne ist notwendig, um korrekte und sinnvolle Anforderungen zu definieren.
- **Zwischenmenschliche Kommunikation:** Anforderungsanalyse ist stark von Kommunikation zwischen Stakeholdern abhängig und erfordert klare Abstimmung.
- **Sich ständig ändernde Anforderungen:** Anforderungen können sich während des Projekts ändern und müssen flexibel angepasst werden können.
- **Wiederverwendbarkeit der Ergebnisse:** Erarbeitete Lösungen sollten so gestaltet sein, dass sie in zukünftigen Projekten wiederverwendet werden können („Eine neu erarbeitete Lösung rechnet sich erst nach drei Projekten!“).

1.3 Techniken

1.3.1 Komplexitätstechniken

- **Abstraktion (prozedurale Abstraktion, Datenabstraktion):** Reduktion auf wesentliche Eigenschaften zur Beherrschung von Komplexität.
- **Kapselung (Information Hiding):** Verbergen interner Details und Zugriff nur über definierte Schnittstellen.
- **Vererbung (Aufbau von Taxonomien):** Ableitung neuer Klassen aus bestehenden zur Wiederverwendung und Strukturierung.
- **Assoziation:** Beziehung zwischen Objekten oder Klassen im System.
- **Nachrichtenkommunikation:** Austausch von Informationen zwischen Objekten durch definierte Nachrichten.
- **Organisationsmethoden:**
 - Objekt / Eigenschaften: Beschreibung eines Objekts durch seine Attribute.
 - Objekt / Bestandteile: Zerlegung eines Objekts in Teilkomponenten (Komposition/-Aggregation).
 - Klassifikation von Objekten: Einordnung in Klassen anhand gemeinsamer Merkmale.
- **Verhaltenskategorien:**
 - Unmittelbare Verursachung: Direkte Ursache-Wirkungs-Beziehung zwischen Ereignissen oder Objekten.
 - Ähnliche Evolution: Ähnliche Entwicklung von Objekten über die Zeit.
 - Ähnliche Funktionalität: Gruppierung aufgrund gleichartiger Aufgaben oder Funktionen.

Beispiel: Objektorientiertes Modell eines Bibliothekssystems

Ein Bibliothekssystem dient zur Verwaltung von Büchern, Benutzern und Ausleihen.

- **Abstraktion:** Ein Buch wird nur durch relevante Eigenschaften wie Titel, Autor und ISBN beschrieben.
- **Kapselung (Information Hiding):** Die interne Speicherung der Ausleihen ist verborgen, Zugriff erfolgt nur über definierte Methoden (z.B. `ausleihen()`).
- **Vererbung:** Es gibt eine allgemeine Klasse *Medium*, von der *Buch* und *DVD* erben.
- **Assoziation:** Ein Benutzer kann mehrere Bücher ausleihen, ein Buch kann zu verschiedenen Zeitpunkten von verschiedenen Benutzern ausgeliehen werden.
- **Nachrichtenkommunikation:** Ein Benutzer sendet eine Anfrage `ausleihen()` an das Bibliothekssystem.
- **Organisationsmethoden:**
 - Objekte werden nach Eigenschaften strukturiert (z.B. Titel, Autor).
 - Ein Buch besteht aus Bestandteilen wie Kapitel.
 - Bücher werden in Klassen wie „Fachbuch“ oder „Roman“ klassifiziert.
- **Verhaltenskategorien:**
 - Ausleihvorgänge folgen einer klaren Ursache-Wirkungs-Kette.
 - Bücher entwickeln sich über Editionen hinweg ähnlich.
 - Bücher einer Kategorie (z.B. Fachbücher) zeigen ähnliche Funktionalität.

1.3.2 Zerlegungstechniken

- funktionale Dekomposition
- strukturierte Analys
- Information Modeling
- objektorientierte Zerlegung

Komplexitäts-kriterium	Zerlegungstechnik			
	Funktionale Dekomposition	Strukturierte Analyse	Information Modeling	Objektorientierte Zerlegung
Abstraktion	✓	✓	✓	✓
Kapselung	✓	✓	✓	✓
Vererbung				✓
Assoziation		✓	✓	✓
Nachrichtenkommunikation				✓
Organisationsmethodik			✓	✓
Verhaltenskategorisierung	✓	✓	✓	✓

Funktionale Dekomposition

- **Hierarchisches Zerlegen in Schritte und Unterschritte (Funktionen, Unterfunktionen, Schnittstellen):** Ein Problem wird in kleinere, klar abgegrenzte Funktionen und Teilfunktionen strukturiert, um es besser beherrschbar zu machen.
- **Erfordert Transformieren in den funktionalen Raum und Rücktransformation:** Der Mensch muss das reale Problem in eine funktionale Modellwelt übersetzen und die Ergebnisse wieder in die reale Bedeutung zurückführen.
- **Darstellung mittels Flussdiagrammen und Struktogrammen:** Die Funktionsstruktur wird grafisch dargestellt, um Abläufe und Kontrollstrukturen besser verständlich und nachvollziehbar zu machen.

Probleme

- mangelnde Genauigkeit und Vollständigkeit der Beschreibung
- einseitiges (rein funktionales) Problemverständnis
- indirekte Abbildung ins Modell
- schlechte Wiederverwendbarkeit

Strukturierte Analyse

- Zerlegung eines Problems anhand der Daten (**Datenflussanalyse**):
 - Datenflüsse, Datentransformation und Datenspeicher
 - Prozessbeschreibung und -randbedingungen
 - Datenlexikon
- **"Verfolgen eines Datenflusses"** schwierig für den Menschen
- Vorteile gegenüber funktionaler Zerlegung:
 - für größere Probleme geeignet
 - komplexere (Daten-)Strukturen möglich

Darstellung mit Datenflussdiagrammen (DFD):

- Knoten stellen Daten dar
- Kanten stellen Transformationen (Prozesse) dar

Moderne strukturierte Analyse mit Kontrollflussdiagrammen (CFD):

- Knoten stellen Ereignisse (Events) dar
- Kanten stellen beteiligte Daten bzw. Kontrollflüsse dar

Merke

Reine Datenflussanalysen konnten sich in der Praxis kaum durchsetzen (Daten wurden lange als "Hilfsgrößen" betrachtet).

Information Modeling

- grundsätzlich datenorientierte Zerlegung, aber unter Verwendung komplexer Datentypen (**Objekte** bzw. **Entitäten**)
 - Entitäten und Attribute
 - Relationen und andere Assoziationen
- **Darstellung mit Entity-Relationship-Diagrammen (ERD)** erweitert zu semantischen Datenmodellen (z.B. logische Beziehungen)
- „Objekt“ wird im Sinn **abstrakter Datentypen** verwendet

Objektorientierte Zerlegung

- Klassifikation von Begriffen bereits durch Aristoteles
- Auswahl von Klassen/Objekten und deren Hierarchie ist trivial (vgl. Klassifizierung von Tieren und Pflanzen)
- Bausteine:
 - **Klassen / Objekte** (enthalten Attribute und Methoden)
 - **Klassifikationsstruktur mit Vererbung** (Generalisierung / Spezialisierung)
 - **Aggregationsstruktur** (Gesamt- / Teilstrukturen)
 - **Kommunikation mittels Nachrichten**

Methode von Abbot zur Bestimmung einer guten Objektstruktur

1. Erstellung einer verbalen Problembeschreibung
2. Identifikation von
 - Objekten (Hauptwörter)
 - Attributen (Eigenschaftswörter, Hauptwörter im Genitiv)
 - Methoden (Zeitwörter)
3. Suche nach geeigneten Basisklassen
4. Vereinfachung und Vervollständigung

Vorteile der objektorientierten Zerlegung

- Umsetzung aller Konzepte zur Beherrschung der Komplexität
- direkte Abbildung des Problems
- klare Trennung zwischen Autor und Anwender
- Verschmelzung von (Zielforschung), Analyse, Design, (Implementierung)
- standardisierte Darstellung mittels der **Unified Modeling Language (UML)**-Werkzeug-unterstützung möglich und sehr sinnvoll

1.3.3 Aktuelle Trends bei der Zerlegung

- **Komponentenorientierte Softwareentwicklung:**
 - Verteilung von Systemen in möglichst enkoppelte Komponenten
- **Aspektororientierte Softwareentwicklung**
 - separate Betrachtung gewisser Eigenschaften (Aspekte) der Software (z.B. Fehlerverhalten, Systemverteilung)
 - Anwendung bewährter Praktiken für diese Eigenschaften
- **Anwendungs-Lebenszyklus-Management**
 - Gleichbehandlung von Anforderungen und Änderungswünschen
 - Verknüpfung von Anforderungen mit Tests und Design/Code

1.3.4 Erstellen von Prototypen

Vorteile

- Beschränkung auf Entwicklungsziel
- Weglassen von Realisierungsdetails
- Freiheit der Entwicklung

Nachteile

- Vollständigkeit und Widerspruchsfreiheit der Analyse nur schwer prüfbar
- nur statische Beschreibung der Anforderungen möglich

Verbesserung durch **prototypingorientierte Analyse**

Arten von Prototypen

- **Vollständige vs. unvollständige Prototypen**
 - **Vollständige Prototypen:** Enthalten alle wesentlichen Funktionen des späteren Systems und sind nahezu produktionsreif.
 - **Unvollständige Prototypen:** Bilden nur ausgewählte Teile oder Kernfunktionen des Systems ab, oft zur frühen Validierung.
- **Breite und flache vs. enge und tiefe Prototypen**
 - **Breite und flache Prototypen:** Decken viele Funktionen oberflächlich ab, jedoch ohne tiefe Detailausarbeitung.
 - **Enge und tiefe Prototypen:** Fokussieren sich auf wenige Funktionen, diese dafür sehr detailliert und vollständig umgesetzt.
- **Revolutionäre vs. evolutionäre Prototypen**
 - **Revolutionäre Prototypen:** Werden verworfen und dienen nur zur Konzeptvalidierung.
 - **Evolutionäre Prototypen:** Werden schrittweise weiterentwickelt und bilden die Basis des späteren Systems.

Beispiel: Entwicklung eines Online-Banking-Systems

Ein Softwareteam entwickelt ein Online-Banking-System und verwendet verschiedene Arten von Prototypen:

- **Vollständiger vs. unvollständiger Prototyp:** Zunächst wird nur die Login- und Kontostandsanzeige (unvollständig) implementiert, später ein fast vollständiges System mit Überweisungen und Historie.
- **Breiter und flacher vs. enger und tiefer Prototyp:** Eine erste Version zeigt viele Funktionen (Konten, Überweisungen, Einstellungen), jedoch nur oberflächlich. Danach wird das Überweisungssystem detailliert und vollständig ausgearbeitet.
- **Revolutionärer vs. evolutionärer Prototyp:** Ein früher UI-Prototyp wird verworfen (revolutionär), während das funktionierende System schrittweise erweitert und weiterentwickelt wird (evolutionär).

Gedächtnisstütze

- **Zielforschung:** Was ist das Ziel?
- **Analyse:** Was ist zu erstellen?
- **Design:** Wie soll das Ergebnis aussehen?
- **Implementierung:** Wie wird das Ergebnis umgesetzt?

1.4 Werkzeuge

Papierbasierte Werkzeuge

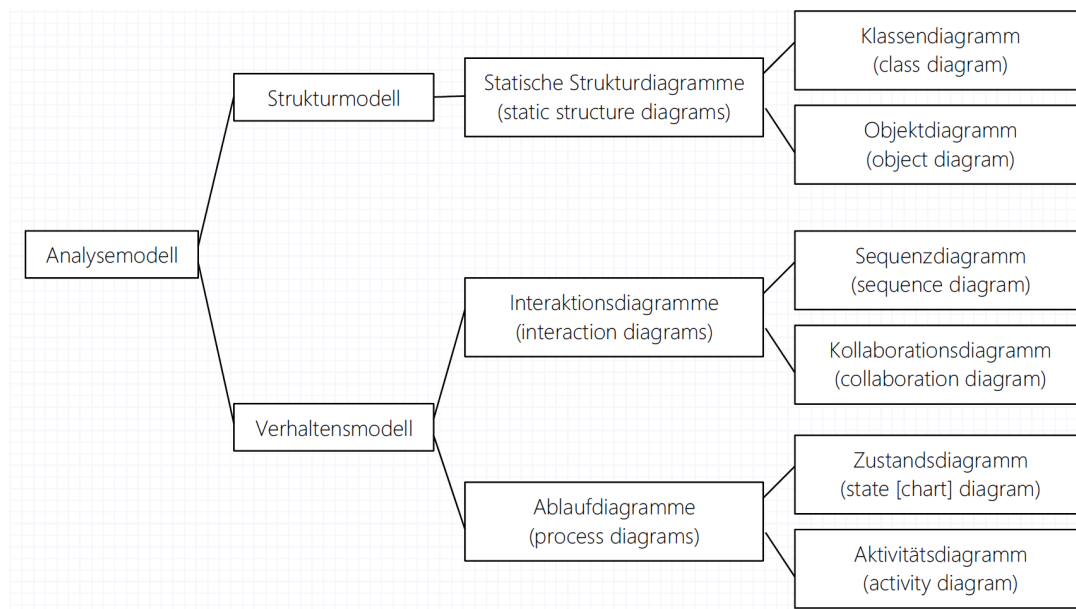
- CRC-Karten
- Haftnotizen-Objekte
- Scripting (textuelle Beschreibung durch Szenarien)

Computerbasierte Werkzeuge

- UML-Werkzeuge

1.4.1 UML

Diagramme der UML für die Analyse



Vorgehen bei der Analyse unter Verwendung der UML

1. Entwickeln des **Strukturmodells**
2. Entwickeln des **Verhaltensmodells**
3. inkrementelles **Verfeinern** der Modelle

1.4.2 Prototyping

Werkzeuge des Prototyping

- **Generatorsystem** bestehen aus Editor und Generator; benutzen formale Spezifikationsprachen . z.B. VDM, Z
- **datenbankbasierte Systeme** (4GL-Systeme, OODB)
- **Hypertextsysteme** (HTML-, XML-basiert)

Kriterien zur Beurteilung von Prototyping-Werkzeugen

- erforderliche Benutzerkenntnisse
- Zeitaufwand bis zum ersten Prototyp
- zu Grunde liegende Benutzerschnittstellen
- Spezifikation der Benutzerschnittstellen
- Verwendung des erzeugten Prototyps
- Unterstützung von Erweiterungen
- Art der funktionalen Erweiterungen
- Entwicklungsumgebung
- Qualität des Prototypingwerkzeugs
- Implementierung des Prototypingwerkzeugs

1.5 Ergebnisse

Ergebnis dokumentation

- Analysebericht
- Lastenheft
- Machbarkeitsstudie

1.5.1 Analysebericht

- Dokumentation der Analyse durch den Auftragnehmer für den Auftraggeber (speziell wenn bereits ein Lastenheft existiert)
- dient als Basis für die Erstellung eines Angebots bzw. für die Einreichung bei einer Ausschreibung

1.5.2 Lastenheft

IEEE: System Requirements Specification - SRS

- ist die Zusammenstellung aller Anforderungen des Auftraggebers hinsichtlich Lifer- und Leistungsumfang
- Es wird also definiert, **WAS WOFÜR** zu lösen ist.

Merke

Lastenheft: Vom Auftraggeber vollständig und widerspruchsfrei zu erstellen, wird in der Praxis jedoch oft vom **Auftragnehmer** erstellt und vom Auftraggeber nur **geprüft und freigegeben**.

Gliederungsvorschlag (gemäß VDI 3694)

1. Einführung in das Projekt, Projektziele
2. Beschreibung des Istzustandes
3. Beschreibung des Sollzustandes
4. Schnittstellen
5. Systemanforderungen
6. Anforderungen für Inbetriebnahme und Einsatz
7. Qualitätsanforderungen
8. Anforderungen an die Projektentwicklung
9. Anhang

Vorprüfung durch den Auftragnehmer auf:

- Machbarkeit
- Vollständigkeit
- Minimalität
- Konsistenz
- Eindeutigkeit
- Plausibilität
- Überprüfbarkeit
- Modifizierbarkeit

1.5.3 Machbarkeitsstudie

- Teilmenge des Lastenhefts, wird **bei unsicherer Machbarkeit** vorab erstellt
- konzentriert sich auf **technische und wirtschaftliche Durchführbarkeit** des Projekts oder definiert ein **machbares Teilprojekt**
- Oftmals ist auch die **personelle Machbarkeit** zu überprüfen.

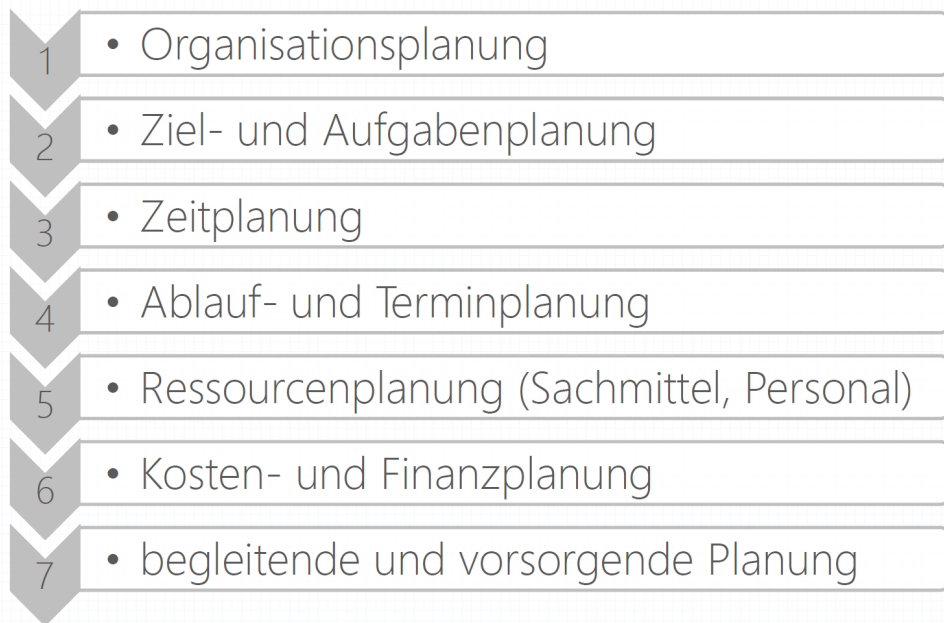
2 Planung

2.1 Ziel

Vorgaben festlegen für

- Projektorganisation
- Aufgaben
- Aufwände
- Termine
- Ressourcen
- Kosten
- Finanzierung
- begleitende Maßnahmen
- vororgende Maßnahmen

2.2 Aufgaben



2.2.1 Organisationsplanung

Planung der Aufbauorganisation

- Teamorganisation (Mitglieder, Führung)
- Kommunikationskonzept (intern, zu AG)
- organisatorische Vorgaben (Normen - z.B. QS, Richtlinien)

Planung der Ablauforganisation

- Prozesse
- Vorgehensmodelle und -methoden
- organisatorische Vorgaben (Normen, Richtlinien; analog zu Aufbauorganisation)

2.2.2 Ziel- und Aufgabenplanung

Zielplanung

Festlegung der Zielwerte inkl. Ausmaß und Termin der geplanten Zielerreichung.

Aufgabenplanung

Gliederung und Schätzung von Teilaufgaben, Festlegen der Aufgabenübergänge (Abnahme)

2.2.3 Zeitplanung

Ermittlung des **Zeitbedarfs** für die Aufgaben bezüglich Schwierigkeit, Sachmittel und Personal mittels Aufwandsschätzung. **Wichtig: Freiraum (slack) einplanen!**

2.2.4 Ablauf- und Terminplanung

auch **Meilensteinplanung** genannt; basiert auf **Aufgaben- und Zeitplan**

- **Ablaufplanung:** Erstellen eines Ablaufgraphen durch Verknüpfung der Aufgaben
- **Terminplanung:** Festlegung von Anfangs- und Endterminen für jede Aufgabe
- **Meilensteine:** wichtige Zwischentermine (Abnahme unter beisein des Auftraggebers)

2.2.5 Ressourcenplanung

auch **Kapazitätsplanung** genannt

- termingerechte Zuteilung von Ressourcen (Sachmittel und Personal)
- Entscheidung von Eigenfertigung oder Fremdbezug

Kapazitätsabgleich: gegenseitige Abstimmung von Ablauf- und Terminplanung mit Ressourcenplanung

2.2.6 Kosten- und Finanzierungsplanung

Kostenplanung
Ermittlung und terminliche Zurordnung von Kosten (asu verplanten Ressourcen)
Finanzierungsplanung
Budgetierung, Steuerung der Finanzmittelbereitstellung (aus Kostenplan)

2.2.7 Begleitende und vorsorgende Maßnahmen

Begleitende Maßnahmen (Planung für **erwünschte** Ereignisse)

- Testplanung
- Installationsplanung, Schulungsplanung
- Qualitätssicherungsplanung
- Berichtswesenplanung, Dokumentationsplanung, ...

Vorsorgende Maßnahmen (Planung für **unerwünschte** Ereignisse)

- Gewährleistungsplanung, Wartungsplanung
- Sicherheitsplanung, Sicherungsplanung
- Notfallplanung, Katastrophenplanung, ...

2.3 Iterativ-inkrementelle Planung beim agilen Vorgehen

2.3.1 Struktur moderner Werkzeuge

- **Planung**
 - Allgemein
 - Mitarbeiter
 - Anforderungen - Aufgaben
 - Iterationen
- **Durchführung**
 - iteration - Anforderungen - Aufgaben
- **Überprüfung**
 - Projekt
 - Mitarbeiter
 - Anforderungen - Aufgaben
 - Iterationen - Anforderungen - Aufgaben

2.3.2 Anforderungen und Aufgaben

Zuordnung

1:n - Beziehung: Jede Aufgabe ist genau einer Anforderung zugeordnet!

Beispiel

- **Anforderung:**
 - Projektverlaufdarstellung
 - Beispiel: Ich möchte den Projektverlauf grafisch dargestellt haben
- **Aufgaben:**
 - Aufwandslinie zeichnen
 - Filter über Aufgaben erstellen
 - Formeln für Auswertekurven festlegen,...

Vergleich

Kriterium	Anforderung	Aufgabe
Phase	Analyse (Was?)	Design (Wie?)
Festlegung	AG	AN
Priorisierung	AG, PV	AN (PL, PV)
Abschätzung	PV [Arbeitstage]	Entwickler (1 Gruppe) [AEH*, h]
Verantwortung	PV (1)	Entwickler (≥ 1)

2.3.3 Planung Projekt

Projekt - Planung - Allgemein:

- Projektname
- Arbeitstage/Iteration
- Arbeitseinheiten/Tag
- Bezeichner für Arbeitseinheiten

2.3.4 Planung Anforderungen

Projekt - Planung - Anforderungen:

- ID (wird fortlaufend vergeben)
- Erstellungsdatum
- Priorität
- Titel
- Beschreibung
- geschätzter Gesamtaufwand (Tage)
- zuständiger Produktverantwortlicher

2.3.5 Planung Aufgaben

Projekt - Planung - Aufgaben:

- ID (wird fortlaufend vergeben)
- Titel
- Beschreibung
- geschätzter Gesamtaufwand (Arbeitseinheiten)
- Aufgabenverantwortlicher
- Typ (intern/intern durchgehend/extern)

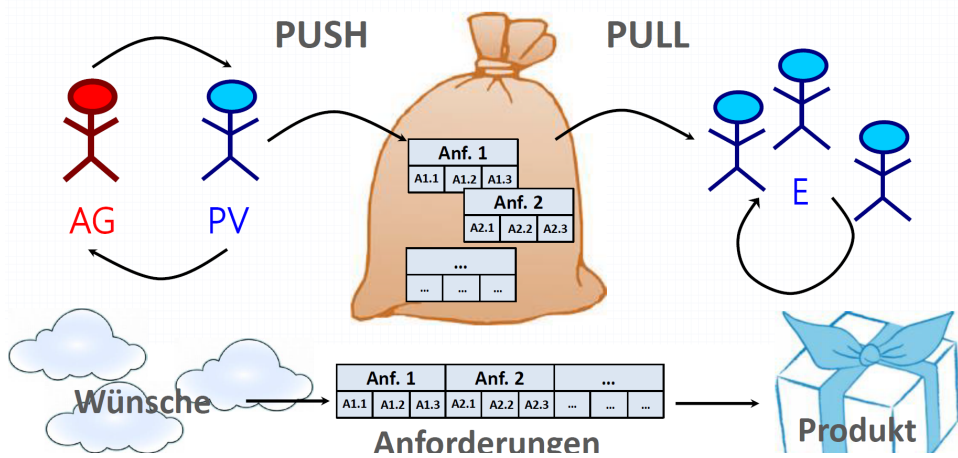
2.3.6 Planung Iterationen

Projekt - Planung - Iterationen:

- Nr. (wird fortlaufend vergeben)
- Arbeitstage (inkl. Beginn, Ende)
- Leistung/Aufwand/Overhead
- zugeordnete Aufgaben (nicht Anforderungen!)

2.3.7 Kombiniertes Push-Pull-Prinzip

Pufferfunktion des Anforderungs- bzw. Iterationskatalogs



2.4 Techniken

Techniken

- Aufwandsschätzung
- Netzplantechnik
- Balkenplantechnik
- Laufendes Schätzen des Restaufwands

2.4.1 Aufwandsschätzung

Verfahren sehr **umstritten** bzgl. Genauigkeit und Sinnhaftigkeit

Beispiele:

- Software Lifecycle Management (SLIM)
- Consolidated Cost Model v. 2 (COCOMO II)
- Function Points (FP)
- Object Points (OP)
- Proxy-Based Estimation (ProBE)

Aufwandsschätzung nach Fausregeln (**Planning Poker, Magic Estimation, etc.**) ist zumindest gleich gut!

2.4.2 Netzplantechnik

Eigenschaften:

- Einteilung in Vorgänge und Ereignisse
- Zuordnung von Zeit und Ressourcen
- Erstellung eines Vorgänger-/Nachfolgergraphen; spezielle Abhängigkeiten: Ende-Anfang (EA), AA, EE, [AE]

Bekannte Beschreibungsmethoden:

- **Metra-Potenzial-Methode** (MPM) heute Atos
- **Critical Path Method** (CPM)
- **Program Evaluation and Review Technique** (PERT)

2.4.3 Balkenplantechnik

- tabellarische Darstellung von Aufwänden durch:
 - Balkendiagramme
 - Histogramme
 - Gantt-Charts
- Eigenschaften:
 - gut für Ressourcenplanung
 - schlecht für komplexe Zusammenhänge

2.4.4 Laufendes Schätzen des Restaufwands

z.B. bei Scrum, jeden Tag erneut; Wert kann auch erhöht werden

2.5 Werkzeuge

Projektplanungssysteme gut zur Dokumentation und zur Planadaptierung

Allgemeine Vorgehensweise:

1. neues Projekt anlegen (Name, Beginn, Kalender, ...)
2. Vorgänge eingeben und gruppieren (Aufgabenplanung)
3. Dauer zuordnen (Zeitplanung)
4. Vorgänge verknüpfen (Ablaufplanung)
5. Meilensteine festlegen (Terminplanung)
6. Ressourcen zuordnen (Ressourcenplanung)
7. Ressourcen- und Terminkonflikte auflösen (Kapazitätsabgleich)
8. Kosten eingeben (Kostenplanung)
9. Kostenüberschreitungen und Finanzierungslücken beseitigen (Finanzierungsplanung)

2.6 Ergebnisse

Interne Ergebnisse:

- Aufgabenplan
- Terminplan
- Ressourcenplan
- Kosten- und Finanzplan

Externe Ergebnisse:

- Meilensteinliste

Gefahr: Möglichkeit eines zukünftigen Schadens, potenzielles Problem

Problem: eingetretene Gefahr -- vorhersehbares Problem: unausweichliche negative zukünftige Konsequenz

Chance: Möglichkeit, Erwartungen zu übertreffen

3 Risikobehandlung Risiko: Auswirkungen von Unsicherheiten auf die Ziele eines Systems

3.1 Definition

Risikobehandlung = proaktive Berücksichtigung wahrscheinlicher Probleme

Situierung der Risikobehandlung in der Projektentwicklung

- Risikobehandlung ist proaktiv
- Auch viele Techniken der Projektentwicklung reduzieren das Projektrisiko, aber reaktiv

Unterscheide:

- allgemeine Risiken
- projektspezifische Risiken

Risikobehandlung nur für projektspezifische Risiken

3.2 Ziele

Ziele:

- vorab Überlegen der Reaktionen auf potenzielle Probleme inkl. Treffen der vorbeugenden Maßnahmen
- Abwägen von Gefahren und Chancen
- quantitatives Bewerten der Risiken zur Auswahl der für die Behandlung sinnvollsten

3.2.1 Notwendigkeit von Risiken

Existierende Produktionsmittel erreichen bessere wirtschaftliche Performanz nur durch größere Unsicherheit, i.e. größeres Risiko

Risiken sind oft **wirtschaftliche Chancen**, daher die richtigen auf sich nehmen.

Ein erfolgreiches Projekt behandelt - die richtigen! - Risiken erfolgreich.

Risikoprofil = Summe der Einzelrisiken eines Projekts

3.2.2 Bestimmung des Risikofaktors

Risikoeigenschaften:

- Wahrscheinlichkeit (Erwartungswert)
- Auswirkung (Schadenskoeffizient)
- Zeitrahmen
- Behandlungsmöglichkeit
- Kopplung sind vom mögl. Problem auch noch andere Projekte abhängig

Risikofaktor = Erwartungswert * Schadenskoeffizient

Die 10 Wichtigsten Risiken bei Softwareprojekten

Top 10-Risikoelemente			
	1980er Jahre [B. Boehm]	1990er Jahre [B. Boehm]	21. Jhdt. [Pare et al., 2008]
1	personelle Defizite	personelle Defizite	Fehlen von Projekt-Champions
2	unrealistische Termin- und Kostenvorgaben	unrealistische Termin- und Kostenvorgaben, ungenügende Beachtung des Prozesses	Fehlende Unterstützung des oberen Managements
3	Entwicklung falscher Produkteigenschaften	Defizite bei zugekauften Komponenten (COTS)	Schlecht verstandener Systemnutzen
4	schlecht gestaltete Benutzerschnittstelle	falsch verstandene Produkteigenschaften	Zweideutigkeiten im Projekt
5	„Vergolden“ (Realisieren unnötiger Eigenschaften)	schlecht gestaltete Benutzerschnittstelle	Abweichen des Systems von lokalen Praktiken und Prozessen
6	schleichende Änderungen der Funktionalitäten	schlechte Architektur, Performanz, Qualität allgemein	Politische Spielereien oder Konflikte
7	Defizite bei zugekauften Komponenten	Entwicklung falscher Produkteigenschaften	Fehlen von erworbenem Wissen oder Fähigkeiten
8	Defizite bei extern vergebenen Aufgaben	Aufbau auf oder Einbeziehung von Legacy-Software	Personelle Änderungen im Team
9	(Echt-)Zeitperformanz	Defizite bei extern vergebenen Aufgaben	Instabilität der Organisation
10	Überfordern der Technologie	Überfordern der Technologie	Ungenügende Ressourcen

3.3 Aufgaben

Risiko-Bewertung

- Risiko-Identifikation
- Risiko-Analyse
- Risiko-Evaluierung

Risiko-Beherrschung

- Risiko-Vorsorgeplanung
- Risiko-Überwachung
- Risiko-Überwindung

3.3.1 Risiko-Identifikation

Ermittlung der relevanten projektspezifischen Risikokriterien

Quellen:

- Analysebericht, Lastenheft, Projektplan
- lesson learned aus ähnlichen Projekten
- unsicher abschätzbare Attribute, ungenaue Schätzwerte
- intelligent diskutieren

Vorgehen:

- **Bottom-up:** Risikoenennung durch (alle) Mitarbeiter; führt zu langen Listen, hoher Konsolidierungsaufwand!

- **Top-down:** Risikoermittlung aus der Bilanz und den Unternehmenskennzahlen; dabei wird leicht Wichtiges übersehen!

Oft wird nicht angegeben, wie unsicher die Angaben in Dokumenten sind!

3.3.2 Risiko-Analyse

- Ermittlung der Schadenswahrscheinlichkeit und des potentiellen Schadens
- durch mehrere Mitglieder in einem Treffen
- am besten quantitativ

3.3.3 Risiko-Evaluierung

- Reihung der Risikoelemente entsprechend ihres Projekteinflusses (Risikofaktor, Hausverstand anwenden)
- Entscheiden ist das Vertrauen in die Zahlen
- Ergebnis: **watch list, Top-n-Liste**

3.3.4 Risiko-Vorsorgeplan

- Festlegung eines Aktivitätenplans
- für jedes behandelnswerte Risiko

3.3.5 Risiko-Überwachung

- Überprüfen der Risiken und Vorsorgepläne auf notwendige Aktivitäten und Planadaptierungen
- periodisch und bei Anlass
- Oft wird nicht rechtzeitig eingestanden, dass ein Risiko zu einem Problem wird!

3.3.6 Risiko-Überwindung

- Anwenden der im Vorsorgeplan festgelegten Technik der Risikobehandlung
- Auftraggeber mit einbeziehen

3.4 Techniken

3.4.1 Abschätzen des Risikoverhaltens

Risikoverhalten von Auftraggeber und Auftragnehmer

(sind risikofreudig, risikoindifferent, risikoscheu?)

Risikoverhalten bestimmt die **Strategie der Risikomitigation (Risikobehandlung, Risikominderung)**

3.4.2 Strategie der Risikomitigation

- **Risiko-Vermeidung:** Elimination eines Risikos durch Wahl eines alternativen Vorgehens (erhöht oft Risiko an anderer Stelle)
- **Risiko-Reduktion:** Verringerung des Risikos nach genauerer Untersuchung der Risikoursache (evtl. durch Aneignung zusätzlichen Wissens)
- **Risiko-Annahme:** bewusste Entscheidung, die Konsequenzen im Falle eines Problems zu tragen (schriftlich begründen und unterweisen: Warnung, Instruktion, Ausbildung)
- **Risiko-Übertragung:** Transfer des Risikos auf andere Verantwortungsbereiche (inkl. Zuständigkeit für Problemlösung)

3.4.3 Mitigation der wichtigsten Risikoelemente

	Risikoelement	Behandlung (Beispiel)
1	personelle Defizite	gutes Personal vertraglich binden
2	unrealistische Termin- und Kostenvorgaben, ungenügende Beachtung des Prozesses	Entwicklung inkrementell durchführen
3	Defizite bei zugekauften Komponenten (COTS)	Qualifikationstests institutionalisieren
4	falsch verstandene Produkteigenschaften	Benutzerdokumentation früh liefern
5	schlecht gestaltete Benutzerschnittstelle	Prototypen erstellen
6	schlechte Architektur, Performanz, Qualität allgemein	Systemrahmen verwenden
7	Entwicklung falscher Produkteigenschaften	Hemmschwelle des AG erhöhen
8	Aufbau auf oder Einbeziehung von Legacy-Software	Designmuster anwenden
9	Defizite bei extern vergebenen Aufgaben	Zwischenergebnisse reviewen
10	Überfordern der Technologie	externe Peers einbeziehen

Tabelle 2: Risikoelemente und mögliche Behandlungsmaßnahmen

3.4.4 Verwenden von Standardprodukten

Standardprodukte (**Commercial-off-theShelf-Produkte - COTS**) können ebenfalls Risiken reduzieren.

Vorteile

- sofort verfügbar, kalkulierbar
- reife Technologie, umfangreiche Unterstützung
- plattformunabhängig/-übergreifend
- kein Wartungspersonal erforderlich

Nachteile

- Abhängigkeit von Zulieferer; Lizenzgebühren zu Beginn
- nicht optimale Ausnützung des Zielsystems
- Beschränkungen (benötigter) Funktionalität
- schwer skalierbar -> Produkt immer groß
- oft inkompatibel zu anderen zugekauften Komponenten

3.4.5 Fehlerbaumanalyse (FTA)

- **Ziel:** Einschätzung der Fehlerwahrscheinlichkeit
- **Zweck:** Systematischer Ansatz zur Gefährdungsanalyse
- **Fragestellung:** deduktiv (Was verursacht ...?)
- **Vorgehen:** Postulierung einer unerwünschten Hauptfolge (Top Event), rekursive Ursachen-suche auf der jeweils nächsten niedrigeren Ebene bis zu den **Blackbox** Komponenten
- **Darstellung:** als Baum; Fehlerkombination mittels UND/ODER

3.4.6 Fehler-Möglichkeiten und Einfluss-Analyse (FMEA)

Failure Mode and Effects Analysis (FMEA):

- **Ziel:** Einschätzung des Schadensausmaßes
- **Zweck:** Systematischer Ansatz zur Untersuchung und Bewertung der Auswirkungen einzelner Fehler (**System-FMEA**); für Prozesse (**Prozess-FMEA**) oder Endanwender (**Anwendungs-FMEA**)
- **Fragestellung:** induktiv (Was passiert, wenn ...?)
- **Vorgehen:** Untersuchung jeweils einer Komponente zu einem Zeitpunkt, Verfolgung der Auswirkungen rekursiv auf der jeweils nächsthöheren Ebene
- **Darstellung:** als Tabelle

Fehlermöglichkeits- und - einflussanalyse				Datum				Prozess/Produkt/System-Name		Erstellt von (Name, Abteilung)					
				15.01.2015				Paketlieferung		M. Meier, Verwaltung					
☐ System-FMEA ☐ Konstruktions-FMEA ☒ Prozess-FMEA				Datum überarbeitet				Prozess/Produkt/System-Nummer		Modell/System/Fertigung					
				10.08.2015				-		Online-Bestellungen					
System/ Produkt / Prozess	Mögliche Fehler			Derzeitiger Zustand				empfolene Abstellmaß- nahmen	Verantwortlich- keit	Verbesserter Zustand					
	Art	Folgen	Ursachen	Kontrollmaß- nahmen	Auftreten	Bedeutung	Entdeckung			RPZ	Kontrollmaß- nahmen	Auftreten	Bedeutung	Entdeckung	RPZ
Paket- lieferung -Produkt aus Lager holen -Produkt verpacken -Paket einladen -Paket transportie- ren -Paket ausliefern	Lieferung verspätet	Kundenun- zufrieden- heit	Probleme mit der Strecke zB. Stau	Verkehrs- funk hören	5	4	7	140	Poutenplanung mit genauerer Berechnung	M. Meier Abt. Verwaltung	Dokument- ation und Messung der Lieferver- spätungen	3	4	5	60
	Paket beschädigt	Schlechte Online- Bewert- ungen	schlechtes Verpack- ungs- material	physikal. Labortest auf Belast- barkeit	3	6	2	36	keine	-	-				0
		Hohe Zahl an Rück- sendungen	Fehler beim Einpacken	Kontrollen beim Einpacken	6	8	7	336	Lehrgang für Mitarbeiter zur Stärkung des Qualitätsbe- wusstseins	Team aus Vertriebsab- teilung	Lehrgang durchgeführt am 02.04.15	3	8	4	96
		Vermehrte Beschwer- den	Schlechte Sicherung im Fahrzeug	regelmäß. Überprüfen des Fahrzeugs	2	8	3	48	keine	-	-				0
		Auftragsver- luste						0							0
								0							

$$\text{Risikoprioritätszahl (RPZ)} = A * B * E$$

- A = Auftretenswahrscheinlichkeit [1-10]
- B = Bedeutung / Schadensausmaß [1-10]
- E = Entdeckungswahrscheinlichkeit [1-10]

3.4.7 SWOT-Analyse

Strengths-Weaknesses-Opportunities-Threats-Analyse (SWOT)

- **Ziel:** Erarbeitung des Schlüsselprofils eines Unternehmens zur Strategieentwicklung (auch im Bereich Risiko)
- **Zweck:** Chancen- und Risikenanalyse um sich der eigenen Stärken und Schwächen bewusst zu werden
- **Positionierung:** im strategischen Management
- **Darstellung:** als Matrix (externe vs interne Analyse)

3.5 Werkzeuge

- eigene Risikomanagement-Werkzeuge (z.B. JPL Defect Detection and Prevention System des NASA Jet Propulsion Laboratory)
- Erweiterung von Projektplanungswerkzeugen (z.B. MS Project)
- Anwendung/Erweiterung von Qualitätsmanagement-Verfahren (z.B. Balanced Scorecard)
- Planer für alternative Projektverläufe (Critical Path Analyzer)
- Tabellenkalkulationsprogramme
- einfache Listen (hauptsächlich schriftlich)

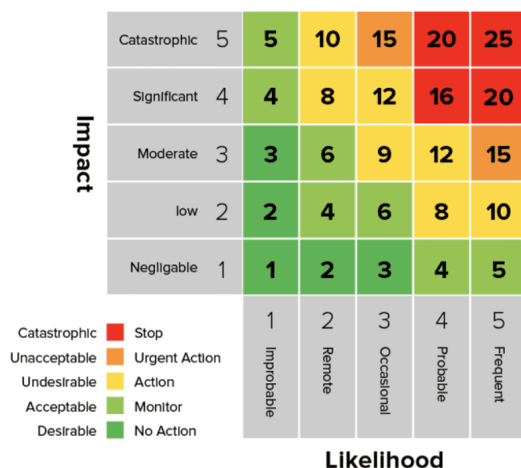
3.5.1 Risiko-Fieberkurve

Bewertung nach Checkliste (z.B. GoDV - Grundsätze ordnungsgemäßer Datenverarbeitung)

3.5.2 Risiko-Hitzekarte

Risiko-Portfolioanalyse mittels Risk Heat/Fever Chart:

visualisiert Risikoelemente in einer 2D-Karte "Wahrscheinlichkeit vs. Schadensausmaß"



3.6 Ergebnisse

- Risiko Top-n-Liste
- Risiko-Vorsorgeplan

3.6.1 Top-N-Liste

Top-N-Liste der Risikoverfolgung: Liste der N wichtigsten identifizieren Risikoelemente, nach Risikofaktor gereiht

Ein Eintrag enthält:

- Beschreibung des Risikoelements
- Risikofaktor
- aktuellen Rang
- vorherigen Rang
- Dauer (wie lange in der Liste)
- Fortschritt bei der Behandlung

Top-N-Liste regelmäßig (und im Anlassfall) **aktualisieren**, vor allem neue und aufsteigende Risiken rasch behandeln!

3.6.2 Risiko-Vorsorgeplan

Der **Risiko-Vorsorgeplan** legt die Vorgehensweise zur Risikoüberwachung und -überwindung fest.

Er enthält:

- zuständiges Personal
- Risiko-Bewertungsprozess
- Risiko-Überwachungsprozess
- Aktualisierungsvorschriften für Top-N-Liste und Vorsorgeplan
- Trigger für außerplanmäßige Aktualisierungen

Der **Risiko-Vorsorgeplan** enthält für jedes relevante Risikoelement:

- Beschreibung des Risikoelements mit Annahmen und Beschränkungen
- Schadenswahrscheinlichkeit und potenzielles Schadensausmaß
- Risiko-Kopplung
- Alternative im Projektplan
- empfohlene Behandlungstechnik mit Arbeitsschritten
- notwendige Einbeziehung des Auftraggebers
- Auswirkungen auf den Projektplan
- Trigger zur Erkennung des Eintretens des Problems
- Kriterien zur Risikoelimination

3.7 Risikomanagement

Risikomanagement = Risikoplanung + Risikobehandlung + Monitoring, Review & Verbesserung der Risikobehandlung

(eigentl. Risikoentwicklung - Begriff aber nicht gebräuchlich)

3.7.1 Motivation

Risikomanagement ist eine Führungsaufgabe!

- zur **Sicherstellung** der Zielerreichung von Organisationen
- zur **Umsetzung** von Sicherheitsanforderungen
- zum rechtzeitigen **Erkennen** (negativer) externer Einflüsse
- zur **Absicherung** externer Erwartungen

3.7.2 Basis Risikopolitik

Die Risikopolitik muss Aussagen über

- Ziele
- Strategien
- Ressourcen für den Umgang mit Risiken

in einer Organisation enthalten.

Weiters sollen in der Risikopolitik Aussagen über

- die Faktoren, die den Erfolg und die Erfolgspotenziale der Organisation beeinflussen
- die Kernrisiken, die die Organisation bewusst eingeht

enthalten sein.

Risiko vs. Sicherheit: Sicherheit =
Security (=Systemsicherheit, von außen geschützt)
+ Safety (Betriebssicherheit, vor internen Fehlern geschützt)

3.7.3 Prozess

Sicherheit = Kompetenz / Komplexität

Situierung des Risikomanagements:

- als integraler Teil des Managements
- eingebettet in die Unternehmenskultur und -praktiken
- maßgeschneidert auf die Geschäftsprozesse der Organisation

3.7.4 Stellen

Risikobeauftragter (Risk Representative):

verantwortlich für das Risikomanagement; benannt von der obersten Leitung

Aufgaben:

- Benennung von Risikoeigner und Risikomanager
- Sicherstellung der operativen Umsetzung der Risikopolitik inkl. Kommunikation und Ressourcen
- Koordination der Aufgaben des Risikomanagementprozesses
- Überwachung des Risikomanagementsystems (auf Eignung, Effektivität und Effizienz)

Risikoeigner (Risk Owner):

Führungskraft, kann Risiken beeinflussen; verantwortlich für seine/ihre Risiken

Risikomanager (Risk Manager)

kann Risiko nicht selbständig beeinflussen; nicht für Risiko, aber für Durchführung des Risikomanagements verantwortlich

4 Design

4.1 Ziel

Design legt fest, **WIE** die Anforderungen für die Implementierung aufbereitet werden.
-> Codieren soll möglichst klar strukturiert und in einfachen Schritten möglich sein!

4.2 Aufgaben

Erstellung des **Designmodells**, das die **physische Sicht** (Aufbau) und **logische Schicht** (Verhalten des zu entwickelnden Systems beschreibt)

Das Designmodell umfasst:

- Systemmodell
- Komponentenmodell

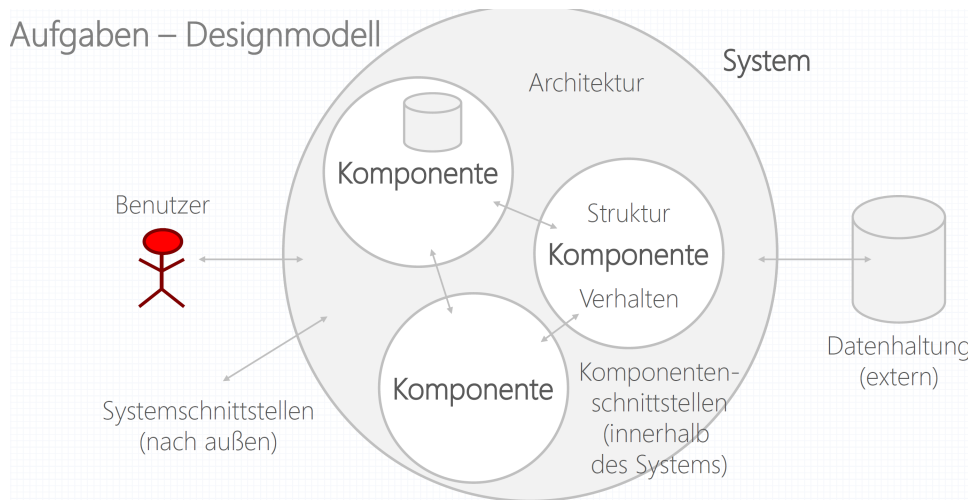
Dementsprechende Aufgaben:

- Erstellen des **Systemdesigns** (System Aufbau, welche Komponenten, grob)
- Erstellen des **Komponentendesigns** (Aufbau dieser Teile des Systems, der Komponenten)

Vorgehen nach **schrittweiser Verfeinerung** (Wirth)

4.2.1 Designmodell

Aufgaben – Designmodell



4.2.2 Gliederung

- **Systemdesign**
 - Architektur
 - Schnittstellen
 - Datenhaltung
- **Komponentendesign**
 - Komponentenstruktur
 - Komponentenverhalten

-> Priorität festlegen, Kompromisse schließen!

Beispiel: Design eines Online-Shops

- **Architektur:** Dreischichtenarchitektur mit Präsentations-, Geschäftslogik- und Datenhaltungsschicht.
- **Schnittstellen:** REST-API für den Zugriff auf Produkte sowie Anbindung an einen Zahlungsdienstleister.
- **Datenhaltung:** Relationale Datenbank zur Speicherung von Benutzern, Produkten und Bestellungen.
- **Komponentenstruktur:** Aufteilung in Benutzerverwaltung, Produktverwaltung, Warenkorb und Bestellabwicklung.
- **Komponentenverhalten:** Nach einer Bestellung prüft das System den Lagerbestand, speichert die Bestellung und versendet eine Bestätigungs-E-Mail.

4.2.3 Modellierung der Architektur

auch **Architekturdesign**; vielfach durch evolutionäre Prototypen

Erweiterung des Strukturmodells

- Entwickeln der Systemarchitektur aus dem Strukturmodell der Analyse (Analysebeschränkungen und Implementierungsvorgaben beachten)
- wiederzuverwendende Komponenten/Untersysteme einplanen

Erweiterung des Verhaltensmodells

- Zerlegung in Untersysteme inkl. deren zeitlichem Zusammenspiel (logische Systemgliederung in physische Untersysteme umsetzen -> verteilte Systeme)
- danach Abbildung auf Prozesse (Nebenläufigkeiten, Echtzeitverhalten beachten!)

4.2.4 Modellierung der Schnittstellen

Systemschnittstellen werden nur durch Aufbau und Protokoll beschrieben (Details in der Implementierung).

Ausnahme: Echtzeitsysteme (Embedded Systems)

Sonderstellung für Benutzerschnittstellen:

- Die vollständige Beschreibung aller Interaktionsabläufe erfolgt bereits im Design.
- Wegen dynamischer Abhängigkeiten ist Benutzerschnittstellen-Prototyping sehr sinnvoll.
- Die Ergonomie der Benutzerschnittstellen (Usability) ist sehr wichtig (vgl. Qualitätsmerkmale für Quality-in-Use).

4.2.5 Modellierung der Datenhaltung

Organisation des Ressourcenmanagements - v.a. sauberer Modellierung der Datenhaltung und -ablage wichtig:

- **Datenstrukturen** im Hauptspeicher
- **Dateien** im Dateisystem / Netzwerk / Cloud
- **Datenbanken** inkl. Anbindung (zentral / verteilt)

4.2.6 Modellierung der Struktur

Festlegung der Komponentenstruktur:

1. zuerst Abhängigkeiten und Schnittstellen zwischen den Komponenten festlegen und Kontrollfluss spezifizieren
2. wieder zu verwendende Komponenten auswählen
3. Komponentendefinition aus Analyse vervollständigen (wenn notwendig Strukturen ergänzen)
4. iterativ Strukturmodell adaptieren (abstrakte Komponenten einführen)

4.2.7 Modellierung des Verhaltens

Festlegung des Komponentenverhaltens:

- Beschreibung der Abläufe (Methoden)
- normalerweise nur **deklarativ**, z.B.:
 - uses: Eingangsparameter
 - changes: Ausgangsparameter
 - informs: erzeugt Nachrichten
 - invariant: Invarianten
 - requires: Vorbedingungen
 - ensures: Nachbedingungen
 - produces: Rückgabewerte
- nur bei schwierigen Algorithmen oder Performanzproblemen **funktionale** Beschreibung
- zusätzlich **Schnittstellenbeschreibung** (in UML Schnittstellenklasse - Interfaces - und Lollipop Notation)

4.3 Techniken

- Erweitern der Analysemodelle
- Erstellen von Prototypen
- Modellgesteuertes Entwickeln
- Endbenutzer-Entwicklung

4.3.1 Erweitern der Analysemodelle

Zweck: Analysemodell vom Problemraum in den Lösungsraum überführen

Vorgehensweise:

- **traditionelles Vorgehen:** vollständige Erstellung des Designs
- **modernes, agiles Vorgehen:** möglichst einfaches Design - aber Achtung auf gutes Design-Grundgerüst (Big-Picture-Design)
- Generalisierung und Promotion von Wissen (erzeugt Taxonomien)
- **Topdown-Design** versus **Bottom-up-Design** - in der Praxis gemischt

Beispiel: Top-down-Design vs. Bottom-up-Design

Ein Team entwickelt einen Online-Shop.

- **Top-down-Design:** Zuerst wird das Gesamtsystem geplant (Startseite, Produktverwaltung, Warenkorb, Bezahlung). Anschließend wird jedes Modul schrittweise in kleinere Teilkomponenten zerlegt und implementiert.
- **Bottom-up-Design:** Zunächst werden wiederverwendbare Komponenten wie Benutzerverwaltung, Datenbankzugriff und Zahlungsmodul entwickelt. Anschließend werden diese zu einem vollständigen Online-Shop zusammengesetzt.

4.3.2 Erstellen von Prototypen

vor allem **experimentelles Prototyping** und **evolutionäres Prototyping** für

- Architekturdesign
- Benutzerschnittstellendesign

Prototyping ist Grundlage aller iterativ-inkrementellen (agilen) Vorgehensmethoden!

Beispiel: Experimentelles vs. evolutionäres Prototyping

Ein Unternehmen entwickelt eine mobile Banking-App.

- **Experimentelles Prototyping:** Es wird ein einfacher Prototyp der Benutzeroberfläche erstellt, um das Design und die Bedienbarkeit mit Testnutzern zu evaluieren. Nach dem Feedback wird der Prototyp verworfen und die eigentliche Anwendung neu entwickelt.
- **Evolutionäres Prototyping:** Zunächst wird eine lauffähige Version mit den Grundfunktionen (Login und Kontostand) entwickelt. Anschließend wird diese Version schrittweise um Überweisungen, Daueraufträge und weitere Funktionen erweitert, bis das fertige Produkt entsteht.

4.3.3 Modellgesteuertes Entwickeln

Model-Driven Development (MDD): reales System wird durch reine Transformationen aus einem Modell erzeugt

Basis: Model-Driven Architecture (MDA); diese ist Teil des UML2-Standards. Bedenke: UML2 ist eine vollständige Programmiersprache!

Vorteile:

- plattformunabhängige Entwicklung
- Simulation auf Modellbasis möglich
- Plattform-Deployment automatisierbar

Vorgehensweise:

1. Erfasse Anforderungen, Modelle und Prozesse in einem plattformunabhängigen Modell (**Platform-Independent Model - PIM**).
2. Lege Rahmenbedingungen als Metadaten fest (**Meta Object Facility - MOF**).
3. Erzeuge daraus ein plattformabhängiges Modell (**Platform-Specific Model - PSM**).
4. Generiere ausführbaren Code vollautomatisch

4.3.4 Endbenutzer-Entwicklung

engl. **End User Development**

auch **Low Code / No Code Development**

vor allem durch Maßschneidern und Befüllen von Systemrahmen (Application Frameworks)

führt vielfach zu schlechten Ergebnissen (Unerfahrenheit der Benutzer bzgl. gutem Design, speziell bei Architektur, aber auch bei Richtlinien zur Benutzerinteraktion)

4.3.5 Wiederverwendung

Development by Investment - Wiederverwendbarkeit muss gegeben sein!

Vorgehensweise:

- Wiederverwenden von Bausteinen
- Abwandeln von Mustern
- Verwenden von Schablonen
- Konkretisieren von Systemrahmen

Wiederverwenden von Bausteinen

= unverändertes Verwenden vorhandener Komponenten:

- Toolkits
- Modulbibliotheken
- Klassenbibliotheken

Abwandeln von Mustern

= Anwenden schematischer Lösungsvorschläge:

- Designmuster
- Analysemuster
- Prozessmuster

Verwenden von Schablonen

= Erzeugen von Klassen/Objekten mittels

- abstrakten Klassen
- Schnittstellenklassen

Komponente = Zusammenstellung von eng gekoppelten Klassen (z.B. JavaBeans); aktive Komponenten besitzen eigene Intelligenz (**intelligent Agent**)

Konkretisieren von Systemrahmen

= Erweitern/Adaptieren einer Sammlung von Objekten, die die Grundfunktionalität einer Anwendung bieten (**generische Anwendung**)

-> ermöglicht Wiederverwendung des Systemdesigns und Kontrollflusses

Beispiel: Arten der Wiederverwendung am Beispiel eines Online-Shops

- **Bausteine:** Eine bestehende Bibliothek zur Benutzerauthentifizierung wird unverändert in den Online-Shop integriert.
- **Muster:** Für die Erstellung verschiedener Zahlungsarten wird das *Factory Pattern* verwendet.
- **Schablonen:** Eine HTML-Vorlage (Template) dient als Grundlage für alle Produktseiten und wird nur mit den jeweiligen Produktdaten gefüllt.
- **Systemrahmen:** Der gesamte Online-Shop wird mit einem Framework wie *Spring Boot* entwickelt, das grundlegende Funktionen wie Routing, Dependency Injection und Datenbankzugriff bereitstellt.

4.3.6 Empfehlungen für ein gutes Design

- Entwickle modulare Systeme
- Eine Klasse, eine Abstraktion; eine Methode, eine Funktionalität
- Befolge das Geheimnisprinzip
- Begrenze die Tiefe von Vererbungsbäumen
- Betreibe sinnvolle Wiederverwendung
- Verwende Designmuster und Systemrahmen
- Halte die ANzahl der Objektinteraktionen gering

4.4 Werkzeuge

- Computergestützte Designwerkzeuge
- Unified Modeling Language (UML)
- Prototyping

4.4.1 Computergestützte Designwerkzeuge

Die meisten Software-Entwicklungssysteme unterstützen auch das Design (sowie Analyse)

- **Forward Engineering:** vom Modell zum Code
- **Reverse Engineering:** vom Code zum Modell
- **Round Trip Engineering:** vom Modell zum Code und zurück

4.4.2 Unified Modeling Language (UML)

Entwickeln des Systemmodells:

1. Implementierungsbeschränkungen festlegen
2. Architektur mit Hardware/Systemsoftware abstimmen (Verteilungsdiagramm)
3. Datenhaltung organisieren; Persistenz! (Verteilungsdiagramm)
4. Nebenläufigkeiten festlegen (Sequenzdiagramm)
5. Benutzerschnittstellen gestalten (Prototypen)

Entwickeln des Komponentenmodells:

1. Komponentenschnittstellen festlegen (**Komponentendiagramm**)
2. Wiederverwendete Komponenten festlegen (**Komponentendiagramm**)
3. Strukturmodell verfeinern (**Klassen-/Objektdiagramm**)
4. Attribute und Operationen ergänzen (**Klassen-/Objektdiagramm**)
5. Gemeinsamkeiten ableiten (**Klassen-/Objektdiagramm**)
6. Abhängigkeiten realisieren (**Klassen- und Kollaborationsdiagramm**)
7. Kontrollfluss festlegen (**Kollaborationsdiagramm**)
8. Steuer- und Zustandsvariablen eintragen (**Klassendiagramm**)
9. Operationen spezifizieren (**Zustands- und Aktivitätsdiagramm**)

Wichtig: Festlegen von **Designkompromissen** und **Designprioritäten** nötig!

4.4.3 Prototyping

- **Beschreibungssprache für Benutzerschnittstellen:** textuelle Beschreibung des statischen Teils einer Benutzeroberfläche
- **Werkzeuge des Compilerbaus:** vielfach ist Programmablauf durch Strom der Eingabedaten gesteuert; Compilerbauwerkzeuge erstellen Parser etc.
- **Grafische Werkzeuge zur Erstellung von Benutzerschnittstellen:** interaktive Eingabemöglichkeiten; oft kann Ablauf bereits simuliert werden

4.5 Ergebnisse

- Designbericht
- Pflichtenheft
- Benutzerdokumentation

4.5.1 Designbericht

- Erweiterung des Analyseberichts um die Designmodelle und um die Prototypen
- normalerweise intern

4.5.2 Pflichtenheft

Pflichtenheft (bei IEEE: System/Software Design Specification - SDS)

- ist die Beschreibung der Realisierung aller Anforderung des Lastenhefts (VDI).
- Es wird also definiert, **Wie** und **Womit** die Anforderungen zu realisieren sind.

Das Pflichtenheft wird **vom Auftraggeber abgenommen**; - bei agilem Vorgehen ggf. Alternativen überlegen (Abnahme von Zwischenprodukten, Protokollen etc.). **erstellt wird es vom AN!**

Gliederungsvorschlag (gemäß VDI)

- wie Lastenheft, nur Adaptierung und Ergänzung um das Kapitel Design

Checkliste für das Pflichtenheft:

zur Vorprüfung durch den Auftragnehmer (auch Lastenheft-Checkliste durchgehen!)

- Kompatibilität
- technische Qualität
- Realisierbarkeit

4.5.3 Benutzerdokumentation

- erste Version bereits während des Designs erstellt, um die Designergebnisse bei der Implementierung leicht zugänglich zu halten
- bereits Review mit dem Auftraggeber möglich

Komponenten

- Allgemeine Produktbeschreibung
- Installationsanleitung
- Anweisungen zur Inbetriebnahme
- Kurzeinführung (Tutorial)
- Benutzerhandbuch
- Referenzhandbuch
- Referenzkarte (Quick Reference Guide)
- Online-Hilfe
- Supplement, Readme-Datei

Checkliste für die Benutzerdokumentation

- Vollständigkeit
- Korrektheit
- Strukturiertheit
- Modularität
- Benutzbarkeit
- Aktualität

Unstimmigkeiten zwischen System und Dokumentation werden vom Auftraggeber am raschesten erkannt und sind kaum zu dementieren.

5 Implementierung - Codieren

5.1 Ziel

1. Umsetzen der Desinergebnisse in **Programmcode (Quellcode)**
-> Codieren des Designs soll möglichst klar strukturiert und in einfachen Schritten möglich sein!
2. Übersetzen (lassen) des Quellcodes in geeigneten Code für das Zielsystem

5.1.1 Aufgaben

Schrittweises, korrektes Umsetzen des Designs:

1. **Umsetzen des Systemdesigns**
2. **Umsetzen des Komponentendesigns**
3. (komponentenweises) **übersetzen** des Quellcodes in Zielcode
4. **Einzeltest** jeder Komponente
5. **Zusammenbau** zum Gesamtsystem

5.1.2 Geschichte der Programmierung

- um 1940: große Maschinennähe (Assembler)
- um 1960: Unterprogramme (erster Abstraktionsschritt)
- um 1970: strukturierte Programmierung
- um 1980: modulatorientierte Programmierung
- um 1990: objektorientierte Programmierung
- um 2000: komponentenorientierte Programmierung
- um 2010: agentenorientierte Programmierung
- um 2020: KI-gestützte Programmierung

5.2 Techniken

- Transformieren des Designs
- Einfügen logischer Bedingungen
- Instrumentieren des Codes
- Kontinuierliche Integration
- Code-Restrukturierung („Refactoring“)
- Erstellen von Testrahmen

5.2.1 Transformieren des Designs

klassische Kernaufgabe der Programmierung

5.2.2 Einfügen logischer Bedingungen

Assertionen sind logische Aussagen, die Bedingungen im Programm entsprechen

- Vorbedingungen
- Nachbedingungen
- Invarianten

Beispiel: Assertionen (Bedingungen in Programmen)

- **Vorbedingungen:** Bedingungen, die vor der Ausführung einer Methode erfüllt sein müssen (z.B. Eingabewerte gültig).
- **Nachbedingungen:** Bedingungen, die nach der Ausführung einer Methode gelten müssen (z.B. korrektes Ergebnis).
- **Invarianten:** Bedingungen, die während der gesamten Laufzeit eines Objekts oder Systems immer gelten müssen.

5.2.3 Instrumentieren des Codes

Verstehen des Codes mit zusätzlichen Codeteilen zum

- Feststellen der Codelokalität zur Laufzeit
- Prüfen der Pfadabdeckung
- dynamischen Analysieren (Profiling)
- Prüfen der Testqualität (Bebuggen, Mutieren)

Hauptprobleme:

- neue Codeteile können zusätzliche Felder enthalten
- neue Codeeteile verändern manchmal das Programmverhalten

5.2.4 Kontinuierliche Integration

engl. **Continuous Integration (CI)**: jeder Entwickler pflegt kleine Codeänderungen laufend ein

Voraussetzungen:

- eine gemeinsame Codebasis
- Übersetzen und Binden auf Knopfdruck
- funktionale Tests hoch automatisiert
- neueste Programmversion laufend zugänglich

-> in der Praxis nur Werkzeug-gestützt sinnvoll möglich (z.B. Apache Ant, CruiseControl, Jenkins)!

5.2.5 Code-Restrukturierung (Refactoring)

Code-Restrukturierung = disziplinierte Änderung eines existierenden Codes zur Designverbesserung ohne Verhaltensänderung

- Erkennen restrukturierungsbedürftiger Codes durch Muster (**bad smells**)

positiv

einleuchtende Muster mit detaillierten Vorschlägen zur Restrukturierung (teilweise sogar automatisierbar, vgl. zb. Eclipse)

negativ

Kommentare werden auch als Zeichen eines schlechten Codes gewertet! (eher missverständlicher Zweck der Kommentierung)

Beispiel: Ursachen für restrukturierungsbedürftigen Code

- **Doppelter Code:** Gleiche oder sehr ähnliche Codeblöcke an mehreren Stellen im System.
- **Schlechte Struktur:** Unklare Modul- oder Klassenaufteilung.
- **Zu große Klassen/Funktionen:** „God Classes“ oder Methoden mit zu vielen Aufgaben.
- **Starke Kopplung:** Komponenten sind zu stark voneinander abhängig.
- **Schlechte Benennung:** Variablen- und Methodennamen sind unklar oder missverständlich.
- **Häufige Änderungen:** Codebereiche, die ständig angepasst werden müssen.

Restrukturierung darf Lauffähigkeit nicht verändern!

- nur lauffähige Programme restrukturieren
- automatisierte Tests verwenden (Test-Driven Development)

Weiteres Problem

Bei unerfahrenen Programmierern steigt Restrukturierungsaufwand überproportional.

Restrukturierung kann Architekturprobleme nicht lösen!

-> Sinnvollste Anwendung zur **Verbesserung des Komponentendesigns**

5.2.6 Erstellen von Testrahmen

Zweck: Erstellen temporärer Programm(teil)e und Daten zu Testzwecken

- Programmierungsumfang kann umfangreich sein (bis zu gleich viel wie der zu testende Code)
- Code von Testrahmen nicht löschen, sondern nur ausblenden

Komponenten von Testrahmen:

- **Stump (stub):** Testcode für gerufene Routine
- **Treiber (driver):** Testcode für rufende Routine
- **Attrappe(mockup, mock object):** Testcode für Komponenten (mock-up wird oft für nichtausführbare Benutzerschnittstellen-Prototypen verwendet)
- **Dummydatei (dummy file):** Datei mit Testdaten

5.3 Testgesteuerte Entwicklung

5.3.1 Idee

Idee (engl. **Test-Driven Development - TDD**): Testfälle werden vor dem entsprechenden Programm(teil) erstellt (eigentlich programmiert)!

Zweistufiges Vorgehen:

1. Erstellen von Systemtests für Anwendungsfälle (idealerweise durch AG)
2. Erstellen von daraus abgeleiteten Komponententests (durch AN)

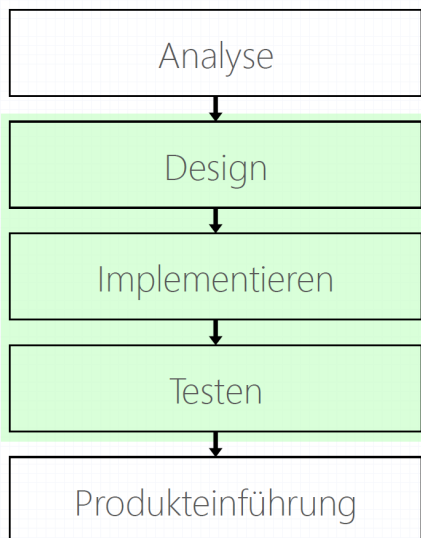
In der iterativ-inkrementellen Entwicklung oft

Ersatz für genaue Produktspezifikation:

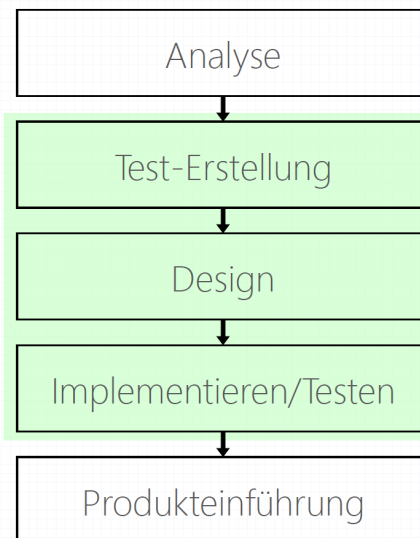
- Produkt erfüllt immer genau die spezifizierten Testfälle.
- Jeder neue Testfall führt zu einer (möglichst kleinen) Produkterweiterung (ggf. Code-Restrukturierung).

5.3.2 Ablauf

Traditionelle Entwicklung



Testgesteuerte Entwicklung



5.3.3 Konsequenz

Testgesteuerte Entwicklung beeinflusst Art des Designs und der Implementierung und ist daher keine Technik des Testens!

5.4 Beispiel

■ MyMath.java

```
package at.scch.examples;

public class MyMath {

    public static int maximum(int x, int y) {
        if (x > y) {
            return x;
        } else {
            return y;
        }
    }

}
```

■ MyMathTest.java

```
package at.scch.examples.tests;

import junit.framework.TestCase;
import at.scch.examples.*;

public class MyMathTest extends TestCase {

    public void testMaximum() {
        int max = MyMath.maximum(4, 3);
        assertEquals(4, max);
    }
}
```

1: Executed object

2: Input values

3: Expected result

4: Actual result

5.4.1 Vorgehen

Testfälle werden jeweils **zu Beginn der Entwicklung** eines Bausteins bzw. Inkrements (Features aus **Kundensicht**) erstellt.

Strategien:

- Starte nicht mit Objekten, starte mit Tests!
- Erstelle Systemtests für Anwendungsfälle
- Daraus Komponententests für dein Design ableiten
- Tests laufend (automatisch) ausführen
- Kleine Inkremente machen und Überblick behalten
- Nach „Red-Green-Refactor“ (Unit Testing) vorgehen

Testfälle werden jeweils auch **während der Entwicklung** auf Basis des Big-Picture-Designs (aus **Entwicklersicht**) erstellt.

5.4.2 Vorteile und Herausforderungen

Vorteile

- Tests sind **automatisch ausführbar** und daher leicht wiederholbar.
- Tests werden vor der Realisierung erstellt -> Erzeugung **testbaren Codes**.
- Tests dienen als **Spezifikation und Dokumentation** („besser als nichts“).
- Die Implementierung wird in jedem Inkrement getestet (**einfaches Regressionstesten**).
- Code-Änderungen sind (auch lokal) rasch überprüfbar.
- Fehler sind rasch lokalisierbar; Folgefehler rasch erkennbar.
- Fehler bei der Code-Restrukturierung werden normalerweise rasch erkannt.
- Man verliert **weniger Zeit mit Debuggen**.
- Zeit für die Testfallentwicklung wird zu Beginn investiert. Damit fällt eine Belastung zu Iterationsende weg. ;-)

Herausforderungen

- **Testfallerstellung** und -auswahl werden **nicht unterstützt**.
- Testfallqualität ist schwer beurteilbar.
- **Testfälle hängen** teils voneinander und auch **von** (geänderten) **Anforderungen ab**. Notwendige Änderungen an Testfällen sind schwer erkennbar.
- Eine große Anzahl von Einzeltests **ersetzen nicht** Integrations- und **Systemtests**.
- Techniken der systematischen Testfallerstellung werden oft vernachlässigt.
- „Einige“ korrekt verlaufene Testfälle „sichern“ die Programm-Korrektheit!

Weitere Erkenntnisse (optimistisch)

- Testklassen sind gleichzeitig Testimplementierung und -dokumentation der zu implementierenden Klassen.
- Testklassen können als Ersatz für eine nicht vorhandene Spezifikation dienen.
- Tester und Entwickler arbeiten enger zusammen.
- Es gibt keinen Code ohne Test.
- Testklassen liefern Sicherheit bei der Code-Restrukturierung.
- Man entwickelt nicht zu viel und nicht zu wenig Code auf einmal.

Weitere Erkenntnisse (pessimistisch)

- Die Implementierung wird eher nach hinten verschoben.
- Man versucht, durch viele, einfache Tests die unangenehmen, komplizierten Tests zu ersetzen.
- Man glaubt, dass ein Testfall, der einmal passt, auch immer passt.
- Eine große Anzahl von Einzeltests ersetzt nicht Integrations- und Systemtests.
- Testgesteuerte Entwicklung verleitet zu Einstellungen wie „Warum denn weitertesten, wenn es eh schon läuft?“.
- **Größtes Problem: Wer testet die Tests?**

Testfallqualität -> Testqualität -> Softwarequalität

5.5 Werkzeuge

Entwicklungsumgebungen, auch **Programmierungsumgebungen** genannt

für Codieren benötigt:

- Editor
- Compiler / Linker
- Versions- und Konfigurationsverwaltungssystem

5.6 Ergebnisse

- Quellcode
- Systemdokumentation

5.6.1 QUellcode

- Gestaltung muss bestimmten Regeln unterworfen werden
- Kommentierung muss Design widerspiegeln

Tipp: Möglichst früh entscheiden, ob Quellcode dem Auftraggeber übergeben wird!

5.6.2 Systemdokumentation

Systemdokumentation (Software/System Reference Manual0)

Zweck:

- Erklärung der Implementierung
- Dokumentation des Know-hows
- Grundlage für Wartung
- Unterstützung der Kommunikation im Entwicklerteam (zusätzlich Entwicklerkollegen als Kontrollleser)

Komponenten

- Beschreibung der Systemstruktur
- Beschreibung des dynamischen Verhaltens
- Beschreibung der Komponenten
- Beschreibung der verwendeten externen Dateinhaltung
- Beschreibung der zentralen Begriffe
- Beschreibung der Installation (Detaillierung des Installationshandbuchs)

Die Systemdokumentation darf **nicht nur Ergebnisse**, sondern muss **auch die Gründe** dafür (**WARUM**) dokumentieren!

6 Implementierung - Debuggen

6.1 Ziel

Finden der Ursache **von Fehlern** und Mängeln **inkl. deren Behebung**
meist vom Codeersteller selbst durchgeführt

6.2 Aufgaben

Sechs Schritte für **effizientes Debuggen**:

1. Fehler stabilisieren
2. Fehler lokalisieren
3. Korrektur ausarbeiten
4. Korrektur durchführen u. dokumentieren
5. Korrektur testen
6. auf ähnliche Fehler prüfen

Unterschied beim (Blackbox-)Testen:

- andere Person testet
- Fehler nicht (genau) lokalisiert
- Fehler nicht behoben

6.3 Techniken

- Statische Programmanalyse
 - Desk Checking
 - Technisches Review
 - Komplexitätsanalyse
 - Strukturanalyse
 - Datenflussanalyse
- Dynamische Programmanalyse
 - Postmortem-Analyse
 - Singlestepping
 - Topdown-/Bottomup-Test
 - Whitebox-Test

6.3.1 Statische Programmanalyse

Das Checking (Schreibtischtest)

manuelles Durchgehen am Schreibtisch, oft anhand konkreter Beispiele

Fachverständnis (Domänenwissen) ist wichtig

Technisches Review

Präsentation von Code in einer Gruppe (First do reviews, then compile!)

Arten:

- **Walkthrough:** Implementieren präsentiert Programm Schritt für Schritt
- **Codeinspektion:** Gruppe prüft Code gegen (verschiedene) Checklisten
- **Codereview:** Gruppe prüft Code vorab und diskutiert ihn mit Implementierer

Komplexitätsanalyse

Bewertung mittels Komplexitätszahlen

Beispiel:

- Lines of Code (LOC)
- Software Sciences (Halstead)
- Cyclomatic Complexity (McCabe)

Viele Maße beruhen auf **keinem realistischen Modell**.

Es wird eher das gemessen, was leicht zu messen ist.

Aber: Eine schlechte Messung ist (meist) besser als gar keine

Strukturanalyse

Finden von Strukturanomalien (z.B. unerreichbarem, totem Code)

Datenflussanalyse

Finden von Datenflussanomalien (z.B. verwendeten Variablen ohne Wert).

Strukturanalyse und Datenflussanalyse oft werkzeugunterstützt

6.3.2 Dynamische Programmanalyse

Postmortem-Analyse (Leichenbeschau)

Auswerten der Ausgaben eines instrumentierten Codes (auch nach Absturz)

Singlestepping

Schrittweises Durchlaufen des Codes mit Hilfe eines Debuggers

Topdown-/Bottomup-Test

Verwendung (von Teilen) des Testrahmens

Whitebox-Test

Programmlauf mit Verfügbarem Quellcode (vielfach mittels Debugger)

6.3.3 Effizienz der Techniken beim Finden von Fehlern

aus einer Studie von Bod Grady

Testart	Gefundene Fehler / h / Person
Reguläre Verwendung (Kunde)	0,21
Blackbox-Test	0,28
Whitebox-Test	0,32
Technisches Review	1,06

Tabelle 3: Fehlerfindungsrate nach Testart

6.4 Werkzeuge

6.4.1 Statische Analysatoren

erstellen z.B. Objektbäume, Aufrufgraphen etc. zur Komplexitätsanalyse

6.4.2 Listengeneratoren

erstellen z.B. Listen von Komponenten, Aufrufen, Kreuzreferenzen, Programmstrukturpläne

6.4.3 Speicherprüfer

ermitteln verdächtige Speicherzugriffe

6.4.4 Testrahmensysteme

Bibliotheken zur Testautomatisierung, die das Schreiben von Testtreibern vereinfachen und vereinheitlichen

- unterstützen ideal die **testgesteuerte Entwicklung**; analog dazu Unterteilung in
 - Testrahmensysteme für Systemtests
 - Testrahmensysteme für Komponententests (*Unit)
- benötigen sinnvollerweise Werkzeuge zur kontinuierlichen Integration
- ersetzen teilweise Debugging (Whitebox-testen), aber keinesfalls das Blackbox-Testen
- bedeuten oftmals hohe Vorabinvestition (Zeit!) für - langfristig - bessere Qualität

6.4.5 Debugger

erlaubt Ablaufverfolgung während des Programmablaufs (meist spezielle Übersetzung notwendig)

Funktionalität:

- Postmortem-Analyse
- Quellcode-Anzeige
- Zugriff auf Laufzeitwerte
- Darstellung von Aufrufketten
- Singlestepping, Setzen von Break Points und Watch Points
- Analyse dynamischer Objektstrukturen
- interaktives Ändern von Werten

6.5 Ergebnisse

6.5.1 Lauffähiges Programm

wird zum Testen (durch andere) freigegeben

6.5.2 Fehlerliste

strukturierte Sammlung begangener oder möglicher Fehler

es gibt Standardlisten, jeder Softwareentwickler soll jedoch seine individuelle Fehlerliste anfertigen (gereiht nach Häufigkeit - **Pareto Prinzip**)

Anwendung:

- zur strukturierten Fehlersuche
- zur Generierung von Lösungsideen
- zur Erstellung projektspezifischer Checklisten
- zur Schwerpunktsetzung bei knapper Testzeit

7 Testen

7.1 Ziel

Testen = systematisches **Aufzeigen von Fehlern** in minimaler Zeit und mit minimalem Aufwand

Man kann nur die Anwesenheit von Fehlern zeigen; Fehlerfreiheit lässt sich niemals zeigen!

-> Jedes Programm enthält Fehler (mentale Irrtumsrate, Halstead)

-> Forderung nach 100% Fehlerfreiheit demotiviert Programmierer!

7.1.1 Ziel aus Stakeholdersicht

Entwicklersicht

Testen ist (aus Programmiersicht) ein destruktiver Prozess!

-> Testen ist psychologisch negativ besetzt.

Kundensicht

Ein erfolgreicher Test deckt einen Fehler auf, bevor ihn der Kunde findet!

-> Aus gesamter Projektsicht ist Testen konstruktiv!

7.1.2 Rollenverteilung

Der Implementierer soll nicht testen!

- Aufgabe ist Fehler zu finden.
- Implementierer ist zu wenig destruktiv.
- Implementierer wiederholt konzeptuelle Fehler beim Testen.
- Andere Testperson motiviert Implementierer
- Implementierer dokumentiert gefundene Fehler nicht.

Der Tester soll nicht korrigieren!

- Tester konzentriert sich auf extern beobachtbares Verhalten.
- Tester kennt Code (oft) nicht.
- Fehlerkorrektur behindert die Fehlersuche
- Fehlerkorrektur bedeutet Mehrarbeit und senkt Fehlerentdeckungsrate.

7.2 Aufgaben

Erkennen von Analyse- und Designfehlern

- Testen von außen bedeutet intensives Arbeiten mit der Benutzerschnittstelle
- Inkonsistenzen und fehlerhafte Bedienungsabläufe sind oft Analysefehler oder Designfehler!

Erkennen von Implementierungsfehlern

- strukturiert suchen (z.B. Versuch alle Fehlermeldungen zu generieren, Testen von Kompatibilität)

Bestimmen des Produktstatus

- Alphatesten
- Betatesten

7.2.1 Bestimmen des Produktstatus

Alphatesten

Testen der einzelnen Funktionalitäten bottom-up

Produkt als Gesamtes noch eher instabil (**Alpha-Version**), Testen einzelner funktionaler Anforderungen ist jedoch bereits möglich

Betatesten

Testen des Produkts als Gesamtes anhand realer Beispielabläufe (**Beta-Version**, nicht unbedingt volle Funktionalität)

- Mitarbeiter des **Auftraggebers** oder speziell eigestellte Betatester fungieren als Testpiloten
-> vorsichtiges Arbeiten, Kooperation mit Entwicklern
- Projektgruppe des **Auftragnehmers** stellt Betatester ab, Auftraggeber liefert geeignete Beispiele -> Vorteil beim Erklären und Beheben von Fehlern, Nachteil beim Einschätzen der Schwere von Fehlern bzw. was als Fehler gesehen wird

7.2.2 Testen der Produktperformanz

Performanztest - Einsatz teilweise bereits während des Debuggens, während des Testens aber aus Anwendersicht

Arten:

- **Laufzeittest** - testet Laufzeitverhalten als Vorgabe für notwendiges Tunen.
- **Speichertest** - testet die Speicheranforderungen.
- **Volumentest** - testet maximal bewältigbaren Aufgabenumfang.
- **Belastungstest** - testet Stabilität bei hoher Systemaktivität.
- **Benchmarkvergleich** - vergleicht Produktversionen.

7.2.3 Vorbereiten der Abnahme

Abnahmevortest

nimmt die Abnahmesituation beim Auftraggeber vorweg (Test der **Abnahmesuite**)

Abnahmesuite wird **idealerweise vom Auftraggeber erstellt**; in der Praxis oft gemeinsam mit dem Auftragnehmer.

7.3 Techniken

- Testfalldesign, Testsuitesdesign
- Generierung und Auswahl von Testfällen
- Testablaufplanung
- Verifikation und Validierung
- Regressionstesten
- Testautomatisierung

7.3.1 Testfalldesign

Ein guter Testfall

- macht Programmfehler offensichtlich
- findet wahrscheinlich einen Fehler
- ist weder zu einfach noch zu komplex.

Testprozess soll möglichst systematisch sein!

Gängige Techniken:

- Äquivalenzklassen bilden
- Grenzwerte analysieren
- Ursache-Wirkungs-Zusammenhänge analysieren

7.3.2 Testsuitesdesign

Erstellen eines Verzeichnisses geeigneter Testfälle (**Testfallsammlung**)

Man kann nie alle Anwendungsmöglichkeiten eines Systems testen!

Gliederung in:

- **Konformanztests** (müssen korrektes Ergebnis liefern)
- **Nonkonformanztests** (müssen korrekte Fehlermeldung liefern)
- **Qualitätstest** (für nichtfunktionale Anforderungen)

Beispiel: Testarten im Software-Testing

- **Konformanztests:** Prüfen, ob die Software den Spezifikationen und Anforderungen entspricht.
- **Nonkonformanztests:** Prüfen bewusst ungültige oder unerwartete Eingaben, um Fehlverhalten zu erkennen.
- **Qualitätstests:** Bewerten nicht-funktionale Eigenschaften wie Performance, Stabilität und Benutzerfreundlichkeit.

7.3.3 Testfallgenerierung

Vorgehensweisen:

- aus Pflichtenheft und Dokumentation extrahieren und Grenzwerte für die Eingabewerte festlegen
- mit Referenzprogramm korrekte Testergebnisse erstellen
- interaktiv mit System arbeiten
- Konkurrenzprodukte zu Vergleichszwecken heranziehen
- korrekte Ergebnisse für spätere Vergleiche aufheben
- alle Ergebnisse immer elektronisch aufheben

7.3.4 Testfallauswahl

Kriterien:

1. Fehler leben in Kolonien -> **Fehler nahe bei Fehlern** suchen
2. für Anwender **verheerende** oder auffällige **Fehler** suchen
3. am **häufigsten verwendete** Programmbereiche ermitteln
4. wesentliche **Produktvorteile** sicher stellen
5. für die Implementierer **schwierigste Bereiche** ermitteln
6. für den Tester **verständlichste Bereiche** auswählen

7.3.5 Testablaufplanung

beginnt spätestens mit Verfügbarkeit des Pflichtenhefts

1. Testvorbereitung
2. Qualifikationstest
3. Testzyklus

Testvorbereitung

Beginn während des Designs, spätestens während der Implementierung

Aktivitäten ohne verfügbares Testprodukt

- Ziel und Aufgaben des Projekts lesen
- Review existierender Dokumente durchführen
- Testdaten des Auftraggebers anfordern und analysieren (wie sind Datensätze aufgebaut etc.)
- Abnahmeprozess festlegen
- Benutzerschnittstelle und -dokumentation reviewen
- Konkurrenzprodukte analysieren

Qualifikationstest

prüfen, ob eine Programmversion stabil genug für Testen ist -> Erstellen einer **Qualifikations-testsuite**, Testen mit **Qualifikationstest**

Vorgehen:

- Qualifikationstest soll nur Hauptbereiche testen.
- Sein Sinn muss den Implementierern klar sein.
- Eine AUtomatisierung diese Tests ist sehr sinnvoll.
- Qulaifaikationstestsuite evtl. Implementierern zugänglich machen.

Testzyklus

Schritte:

1. Start with a bang! - Implementierer rasch mit Fehlerbehebungsaufträgen versorgen
2. Produktstabilität und -zuverlässigkeit abschätzen (Prognose für notwendige Testzyklen)
3. Offensichtliche Fehler aufdecken (Simulation des ersten Arbeitstages beim Anwender, Inkonsistenzen zur Dokumentation)
4. Testsuite(n) durchlaufen
5. Ergebnisse dokumentieren (möglichst elektronisch und geeignet für automatische Wiederholung/Prüfen)
6. Testsuite(n) adaptieren

Inkrementelles Testen ist Big-Bang-Testen auf jedem Fall vorzuziehen!

7.3.6 Regressionstesten

auch: **Resttesten**

Zweck:

- Überprüfung der Fehlerkorrektur
- Absicherung gegen neu eingeschleppte Fehler

mittels **Regressionstestsuite**, die laufend (möglichst automatisch) abgetestet und adaptiert wird

Regressionstestsuite soll möglichst klein gehalten werden (redundante Tests entfernen, Tests kombinieren).

7.3.7 Testautomatisierung

Manuelles Testen ist fehleranfällig -> müsste selbst laufend getestet werden!

Testautomatisierung:

1. Testfälle erzeugen (-> Programmverifikation!)
2. Testfälle durchlaufen
3. Ergebnisse überprüfen
4. Testresultate visualisieren
5. Testresultate protokollieren

Testautomatisierung benötigt **Testrahmensystem für Systemtests** -> oftmals hohe Amortisationskosten und lange Amortisationszeit

weitere Risiken

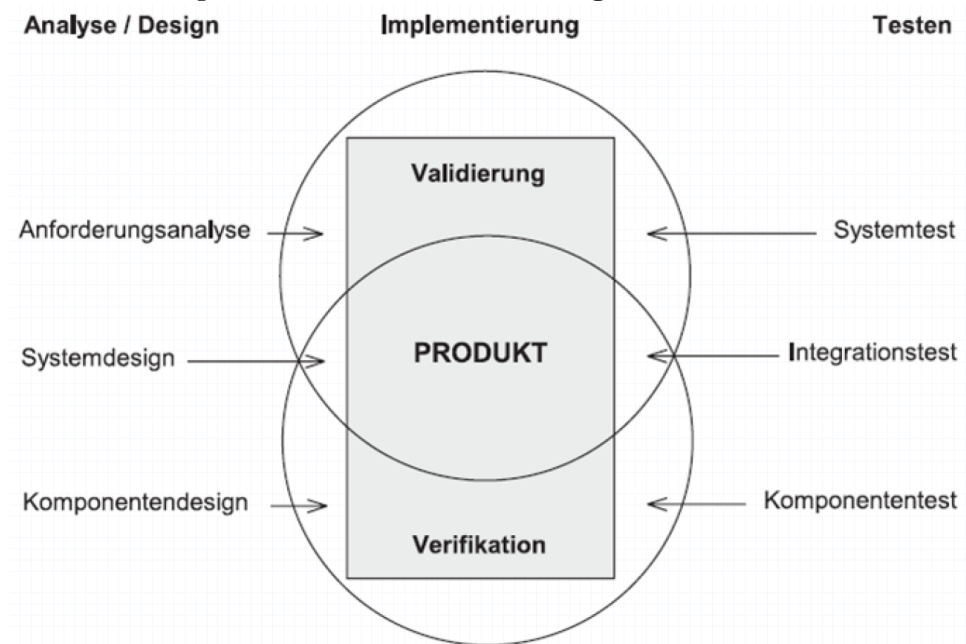
- zu viele simple Testfälle
- wenig neue Testfälle
- Einlernaufwand unterschätzt
- Ergebnisse schlecht analysiert

7.4 Verifikation vs. Validierung

Anforderungen vs. Wünsche

- Was der Kunde sagt (Wünsche) $\neq$ was der Kunde wirklich braucht (Anforderungen)
- Nicht alle Anforderungen werden explizit genannt (explizite vs. implizite Anforderungen)
- Systematische Erhebung und Bewertung der Anforderungen ist notwendig
- Methoden wie **Quality Function Deployment (QFD)** helfen dabei (Was/Wie-Trennung, *House of Quality*)

Zusammenhang Verifikation & Validierung



7.4.1 Verifikation

Testen der Funktionalität (Haben wir das Produkt richtig erstellt?)

- > Überprüfung, ob ein Softwareprodukt seiner Spezifikation entspricht
daher auch **Funktionalitätstest**

Techniken:

- Einzeltest (Unit-Test)
- Modultest, Programmtest
- Integrationstest (Testen des Zusammenbaus)

7.4.2 Validierung

Testen des Gesamtsystems (Haben wird das richtige Produkt erstellt?)

-> Überprüfung, ob ein Softwareprodukt den realen Anforderungen genügt (**Brauchbarkeit**)
daher auch **Systemtest, Gesamttest**

Techniken:

- Subsystemtest (Testen weitgehend unabhängiger Subsysteme und deren Schnittstellen)
- Systemtest, Integritätstest (Testen auf Einsetzbarkeit und Brauchbarkeit bei Anwender)
- (Benutzer-)Akzeptanztest (Testen der realen Verwendung)
- Zertifizierung (wenn gewünscht / erforderlich)

7.5 Werkzeuge

Derzeit oft im **Open-Source-Bereich** verfügbar.

Grund: Projekte unterscheiden sich in späteren Phasen (wie dem Testen) stärker voneinander -> Marks schwierig!

Werkzeuge zur Testunterstützung:

- Dateivergleichswerkzeuge
- Formatübersetzer
- Bildschirmspeicherprogramme
- Monitorprogramme
- Testrahmensysteme
- „Capture & Replay“-Systeme
- Fehlermeldesysteme
- Diagnoseprogramme
- Standardüberwachungsprogramme

7.6 Ergebnisse

- Testplan
- Testssuite
- Testliste
- Fehlerbericht
- Fehlerlogbuch

7.6.1 Testplan

Testplan = Beschreibung von Aufgabenumfang, Vorgehensweise, Ressourcen und Ablauf der beabsichtigten Testaktivitäten

Bedeutung ist of Auftraggeber nicht klar!

Erstellung **benötigt viel Zeit**:

- parallel zu Design und Implementierung durchführen
- auf Pflichtenheft aufbauen
- laufend aktualisieren

Inhalte:

- Beschreibung von Produktziel und Testziel
- Festlegung des Testumfangs (auch was **nicht** getestet wird)
- Auflistung der abgedeckten und nicht abgedeckten Risiken
- Beschreibung der Teststrategie, des Testansatzes und der Testkriterien
- Beschreibung der Testumgebung
- Planung des Testablaufs einschließlich der benötigten Ressourcen
- Festlegung der zu liefernden Testdokumentation

7.6.2 Testsuite

Testsuite = nach Testklasse geordnetes Verzeichnis von Testfällen

Einteilung in:

- funktionale Tests (Konformanztests, Nonkonformanztests)
- nichtfunktionale Tests (Qualitätstests)

7.6.3 Testliste

angefertigt für persönlichen oder entwicklerinternen Gebrauch, erstellt aus dem Pflichtenheft sowie aus Benutzer- / Systemdokumentation

Beispiele für Inhalte von Testlisten:

- Funktionalitäten
- Fehlermeldungen
- im Programm enthaltene Dialoge (Eingabe, Ausgabe, Dateien)
- gelieferte Dokumente
- zum Betrieb notwendige Hardware und Software
- vom System unterstützte Hardware und Software

7.6.4 Fehlerbericht

Fehlerbericht = standardisierte Beschreibung eines entdeckten Fehlers mit Angabe zur Fehlerbehebung

Fehler muss reproduzierbar sein!

Fehlerbericht immer sofort beim Auftreten eines Fehlers schreiben (Fehlerverhalten noch auf dem Bildschirm sichtbar)

7.6.5 Fehlerlogbuch

Fehlerlogbuch = zeitlich geordnete Auflistung aller gefundenen Fehler inklusive ihrer Behebung (auch: **Testlogbuch**)

8 Produkteinführung

8.1 Ziel

Produkteinführung = Abschluss eines Projekts durch die erfolgreiche Aufnahme des **Betriebs** eines Softwareprodukts **beim Auftraggeber/Kunden**

für **Auftragnehmer**: Know-how-Erweiterung durch **Nachanalyse**

In Betrieb befindliche Software benötigt **Wartung und Pflege!**

8.2 Aufgaben

1. Lieferung
2. Installation und Inbetriebnahme
3. Schulung
4. Abnahme und Abschluss
5. Evaluierung
6. Archivierung
7. Wartung (und Pflege)

8.2.1 Lieferung

umfasst **Übergabe und Übernahme** von Produkt inklusive Dokumentation

Der AG erwartet bei der Lieferung ein **brauchbares Produkt**

-> **erster Eindruck** beeinflusst (positive) Einstellung

Oft muss **Zuverlässigkeit** abgeschätzt werden (vom Projektleiter):

1. hohe Zuverlässigkeit
2. mittlere Zuverlässigkeit
3. niedrige Zuverlässigkeit
4. unbekannte Zuverlässigkeit (am schlechtesten)

-> Zumindest **minimale Zuverlässigkeit** schriftlich festlegen.

Projektleiter muss Unternehmensleitung **über Risiken klar informieren!**

8.2.2 Installation und Inbetriebnahme

Installation = Einrichtung eines Produkts in der (simulierten bzw. echten) Zielumgebung (noch ohne es auszuführen)

Aufgaben:

- **Überprüfung** der Zielhardware
- **Konfiguration** von Hardware und Software
- kundenspezifische **Installation**
- **Prüfung** auf Funktionsfähigkeit des Geräts (ohne neue Software)

Inbetriebnahme = Vorbereitung für den Regelbetrieb (erste Ausführung des Produkts)

Aufgaben:

- **Starten und Testen** der Komponenten
- Erstellen von **Testausdrucken**
- **Prüfung auf Funktionsfähigkeit** des Geräts (mit neuer Software)

Vor und nach der Installation ein **Backup** anfertigen!

Auch **Wiederinbetriebnahmetest** durchführen.

8.2.3 Schulung

Schulung = Einweisung in geeignete Verwendung

Bediener haben verschiedene **Rollen** (vgl. **Personas** aus dem Design!)

- **Benutzer (user)** braucht Anwendungsfunktionalität
- **Betreuer (administrator)** braucht Systemfunktionalität, aber auch als Benutzer schulen!

Schulung möglichst **vor der Abnahme** durchführen! -> Sie kann verrechnet werden (Angebot!) und erleichtert die Abnahme.

8.2.4 Abnahme und Abschluss

Abnahme = rechtskräftige Annahme eines Produkts durch den AG; - führt automatisch zum **Abschluss**!

Abnahme dauert **max. 1 Tag** - immer ein Kompromiss zwischen optimalem und akzeptablem Produkt (schriftliche Dokumentation - **Protokoll!** - wichtig)

in der Praxis **erst** sinnvoll **nach Inbetriebnahme** (und Schulung)

8.2.5 Evaluierung

Evaluierung = auftragnehmerinterne **Nachanalyse und Nachkalkulation** (Soll-/Ist-Vergleich des Projektaufwands)

-> daraufhin Adaptierung der Planungsparameter

Projekt wird kostenmäßig abgerechnet, die Zeitplanung normalisiert.

8.2.6 Archivierung

wichtig für Wartung und Wiederauffinden von know-how

Tätigkeiten:

- intern elektronische und Papierablagen zusammenräumen
- gesamtes Know-how dokumentieren
- auftraggeberspezifische Informationen **löschen**

8.2.7 Wartung

Warung (+ Pflege) = Produktverbesserung nach dem Abschluss

Jede Software erfordert Wartung (jedoch anderer Wartungsbegriff als bei materiellen Produkten, da keine Abnutzung).

Wartung im weiteren Sinn =

- **Wartung** im engeren Sinn (**korrigierende Tätigkeiten**)
- **Pflege** (**erweiternde Tätigkeiten**)

Aktivitäten der Wartung (im engeren Sinn):

- **Stabilisierung** (Beseitigung enthaltener und neu eingebrachter Fehler, ereignisgesteuert)
- **Optimierung** (wenn zur bestimmungsgemäßen Verwendung nötig)

Aktivitäten der Pflege:

- **Anpassung** (bei Bedarf; enthält auch Optimierung über die Anforderungen hinaus)
- **Erweiterung** (längerfristig planbar)

Pflegeaktivitäten umfassen **60-80%** der Gesamttätigkeit!

Beispiel: Wartungs- und Pflegeaktivitäten

- **Wartung – Stabilisierung:** Beheben eines Programmfehlers, der unter bestimmten Bedingungen zum Absturz der Anwendung führt.
- **Wartung – Optimierung:** Verbesserung der Datenbankabfragen, sodass Suchanfragen deutlich schneller ausgeführt werden.
- **Pflege – Anpassung:** Anpassung der Software an ein neues Betriebssystem oder eine neue Version einer Datenbank.
- **Pflege – Erweiterung:** Hinzufügen einer neuen Funktion, z.B. eines Exportes von Berichten als PDF.

Einteilung im Englischen:

Wartung + Pflege = **Maintenance**

- **Corrective Maintance** (Stabilisierung, 17%)
- **Adaptive Maintance** (Anpassung, 18%)
- **Perfective Maintance** (Optimierung, Erweiterung 65%)

Kosten für Wartung und Pflege:

Wartung/Pflege macht **mehr als 50% der Gesamtkosten** eines Produkts (über die gesamte Entwicklungs- und Lebenszeit) aus.

- Eine seriöse Kostenschätzung ist nur bedingt möglich.
- In der Praxis werden vielfach 7-15% der Entwicklungskosten pro Jahr kalkuliert.
- Wartungsfreundlichkeit ist wichtig.

Wartungs- und Pflegeaktivitäten sollten voneinander getrennt werden (unterschiedliche Charakteristika) - in der Praxis jedoch schwierig:

- **Wartung nach Aufwand** (Stunden) verrechnen
- **Pflege als Folgeprojekt** kalkulieren (Fixpreis)

8.2.8 Wartungsalternativen

Wartungsdauer (End-Of-Life-Problematik):

Bei alten Produkten ist zu entscheiden:

- **Wartung** (Instandhaltung)
- **Sanierung** (Instandsetzung, Reengineering)
- **Evolution** (Weiterentwicklung, Pflege im Großen)
- **Migration** (Übertragung)
- **Einstellung** (durch neues Produkt ersetzen)

8.3 Techniken

8.3.1 Umstellung

Übertragung von Datenbeständen in die neue Umgebung

Strategien:

- **Direkte Umstellung:** Wechsel vom alten aufs neues System zu einem bestimmten Zeitpunkt (vor allem bei **nicht duplizierbaren Ressourcen**)
- **Parallele Umstellung:** gleichzeitiger Betrieb des alten und des neuen Systems über einen gewissen Zeitraum, dabei laufende Synchronisierung des Datenbestands (bei **sicherheitskritischen Systemen**)
- **Testweise Umstellung:** gleichzeitiger Betrieb des alten und des neuen Systems über einen gewissen Zeitraum mit ein-/mehrmaliger Duplizierung der Daten, ohne regelmäßige Synchronisation

8.3.2 Probetrieb

Evaluierung des Produkts durch den Auftraggeber

Arten:

- Installationstest
- Leistungstest (Benchmarktest)
- Pilottest

Auftraggeber darf kein fehlerfreies, sondern lediglich ein brauchbares System erwarten!

8.3.3 Abnahmetest

Gesamttest unter realen Einsatzbedingungen - prüft (**ideal**) Produkt gegen **Lastenheft**, **real** Produkt gegen aktuelle **Kundenwünsche**

- **Abnahme(test)** ist kein punktuellere Ereignis - genügend Zeit einplanen; Auftraggeber jedenfalls einbeziehen
- **Abnahmesuite:** Menge vorher festgelegter Testfälle - muss vom Auftraggeber dem Auftragnehmer rechtzeitig (VOR der Abnahme!) übermittelt werden
- **Abnahme-Schlussbesprechung:** Abnahme erfolgt, wenn entdeckte Fehler tolerierbar oder nachbesserbar sind. Vollständiges Testen ist niemals möglich!

8.3.4 Nachkalkulation

Soll-/Ist-Vergleich der Planwerte, um die Ursache der Abweichungen zu ermitteln (vgl. Abschnitt **Kontrolle**)

Verglichen werden:

- Anforderungen (Aufgaben)
- Aufwände, Termine
- Ressourcen
- Kosten, Finanzen

8.3.5 Kontinuierliche Lieferung/Freigabe

Englisch: **Continuous Delivery / Continuous Release**

Bei vielen neuen Softwaresystemen (z.B. Web, Mobile Apps)

Eigenschaften:

- aktueller Inhalt (content) untrennbar mit Anwendung verknüpft
- Übergang von Entwicklung zur Wartung kaum terminisierbar
- Produkt wird regelmäßig aktualisiert (kontinuierlich freigegeben)

Hilfreich:

- inkrementelles Prozessmodell (agiles Vorgehen)
- kurze Freigabezyklen (wenige Tage)

Herausforderungen bei Kontinuierlicher Freigabe:

- oft muss Anwendung durchgehen verfügbar sein (24/7)
- Wartung und Pflege im laufenden Betrieb schwierig

Empfehlungen:

- Konfigurations-/Versionsverwaltungssysteme verwenden (Änderungshistorie!)
- Entscheidungen schriftlich festhalten

8.3.6 Development & Operations (DevOps)

Verschränkung von Entwicklung (**Development**) und Betrieb (**Operations**)

- **Anforderungen** (Requirements) und **Änderungswünsche** (Change Requests) werden zunehmend gleich betrachtet und behandelt!
- **Übertragung der Betriebsfähigkeit und -qualität** auf den **Auftragnehmer**
- Ausweitung des Agilen Vorgehens auch auf den Betrieb (Aigle System Administration, Agile Operations; -> **Application Lifecycle Management**)

Begriff DevOps:

- eingeführt 2009 von Patrick Debois (Belgien)
- Definition noch unscharf

8.3.7 Alternative zur Wartung

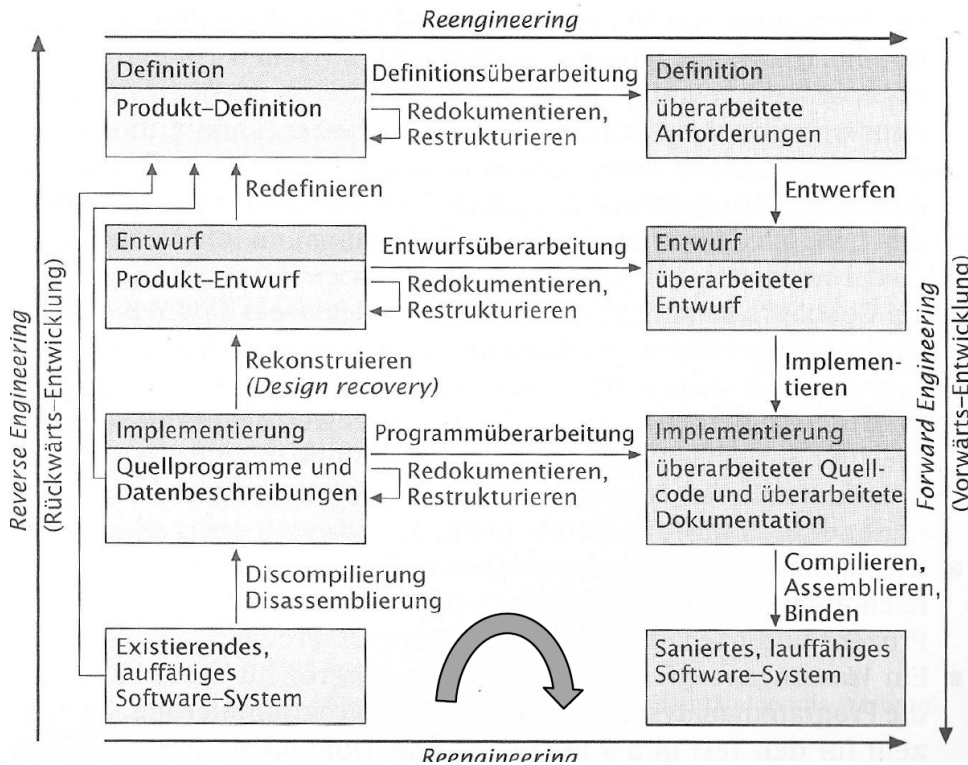
Technik-Auswahl je nach (technischer) **Qualität** und (wirtschaftlicher) **Bedeutung**:



8.3.8 Sanierung

- Renovieren (Reengineering)
- Verpacken (Wrapping)

Reengineering Reengineering umfasst **Reverse Engineering** und **Forward Engineering**:



Wrapping Wrapping (Verpacken) ist **auf allen Abstraktionsebenen** (Attribut/Methode bis hin zur Komponente) möglich:

- **Whitebox-Wrapping** (Verstehen des Systems bis in Innerste; Verpacken aus Kompatibilitätsgründen)
- **Blackbox-Wrapping** (Verstehen nur des Ein-/Ausgabeverhaltens des Systems)
- **Greybox-Wrapping** (Mischform)

8.3.9 Evolution

Evolution in der Natur:

- Nachkommen
- Varianten
- Selektion

Evolution in der IT:

- Versionen
- meist nur eine Revision
- Selektion nicht möglich

Daher in der IT: **Evolution** = Weiterentwicklung, Pflege im Großen; Abgrenzung zu Wartung (maintenance).

Evolution über die gesamte Entwicklung und den Betrieb: **(Anwendungs-)Lebenszyklus** (analog zum Menschen)

Beispiel: Einsatz von Wartungstechniken

1. **Migration:** Umstellung einer Desktop-Anwendung von Windows auf Linux oder in eine Cloud-Umgebung.
2. **Sanierung:** Überarbeitung einer Altanwendung (Legacy-System), um veralteten Code zu bereinigen und die Wartbarkeit zu verbessern.
3. **Wartung:** Behebung eines Fehlers in der Berechnung der Mehrwertsteuer nach einer Kundenmeldung.
4. **Evolution:** Erweiterung einer bestehenden Shop-Software um eine neue Funktion, z.B. die Unterstützung von Apple Pay oder Dark Mode.

8.4 Werkzeuge

Spezielle Werkzeuge existieren nur beschränkt (z.B. **Erweiterungen von Vontinous-Integration-Tools** wie Jenkins oder AntHill bzw. **DevOps-Tools** (z.B. Docker)).

Eingesetzt werden beispielsweise:

- Versionsverwaltungswerkzeuge
- Installationsprogramme
- Simulatoren
- Monitorprogramme
- Schnittstellenprüfer
- Logfile-Ersteller
- Protokolliersysteme

8.5 Ergebnisse

- Betriebsversion
- Installations- und Inbetriebnahmeanleitung
- Abnahmeprotokoll
- Abschlussbericht
- Projektarchiv

8.5.1 Betriebsversion

erste abgenommene Version inklusive Dokumentation (**keine Vollversion!**)

Wichtig:

- korrekte Version der Dokumentation beilegen
- Testdokumentation beilegen
- Readme-Dateioen nur im Notfall beilegen
- Benutzer über Fehler informieren

8.5.2 Installations- und Inbetriebnahmeanleitung

dokumentiert chronologisch **Installation und Inbetriebnahme** (durch Auftragnehmer)

Auftraggeber führt **Bedienerprotokoll**

8.5.3 Abnahmeprotokoll

dokumentiert detailliert die **Abnahme**

wird **unabhängig vom Erfolg** der Abnahme erstellt (Teilabnahmen sind möglich)

Ist ein gemeinsames Protokoll nicht möglich (zu unterschiedliche Ansichten), so werden **Sachverhaltsdarstellungen** erstellt.

8.5.4 Abschlussbericht

interner Bericht des Auftragsnehmers, von allem Mitarbeitern erstellt (manchmal auch als Protokoll einer **Abschlussbesprechung**)

schließt Informationen der Nachkalkulation und der Qualitätssicherung ein

8.5.5 Projektarchiv

dient als **Basis für Wartung und Dokumentation** des erarbeiteten Know-hows

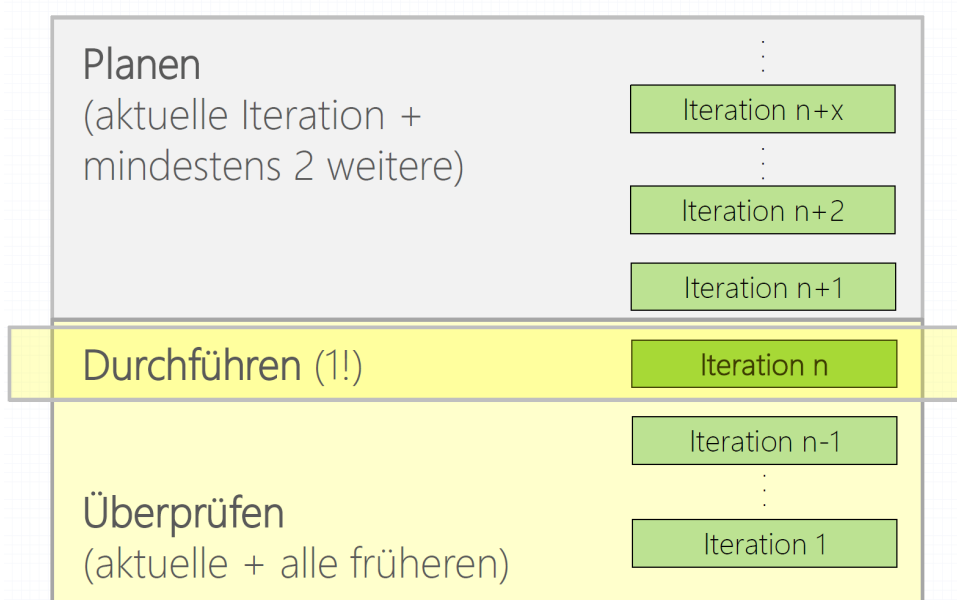
Wichtig:

- alle notwendigen Werkzeuge in den richtigen Versionen mit archivieren
- rechtliche Klärung der Speicherung auftraggeberspezifischer Daten ist notwendig

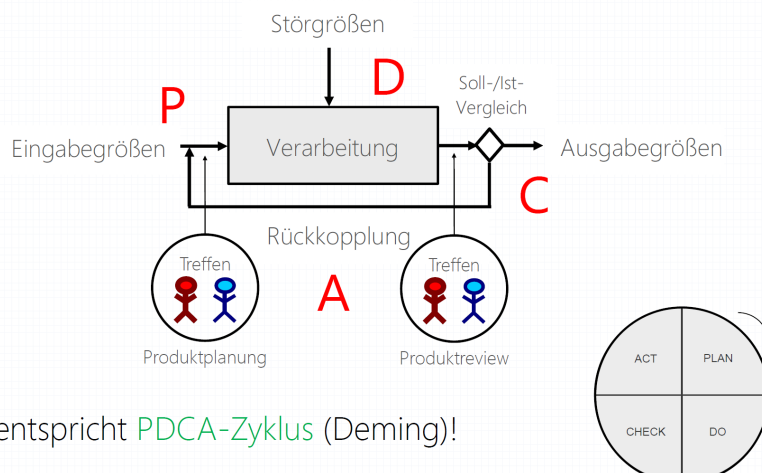
9 Tipps für die agile Softwareentwicklung

9.1 Denken in Iterationen

9.1.1 Konzept



9.1.2 Geregelter Prozess



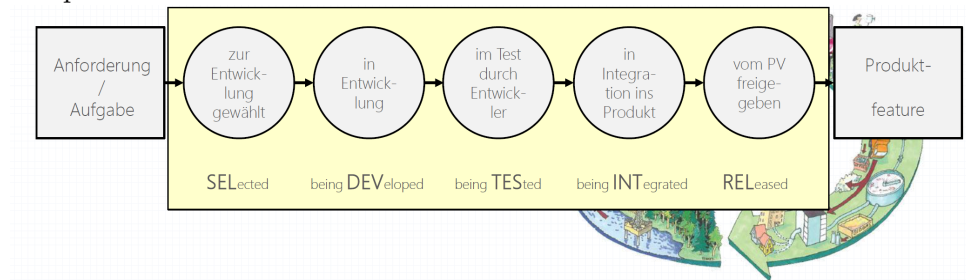
Vorgehen entspricht **PDCA-Zyklus** (Deming)!

9.2 Denken in Zuständen

Aufgaben befinden sich während der Abarbeitung in definierten Zuständen.

- Begriffe frei wählbar;
- Abfolge wichtig (**Zustandskette** bzw. **Wertschöpfungskette**)

Beispiel:



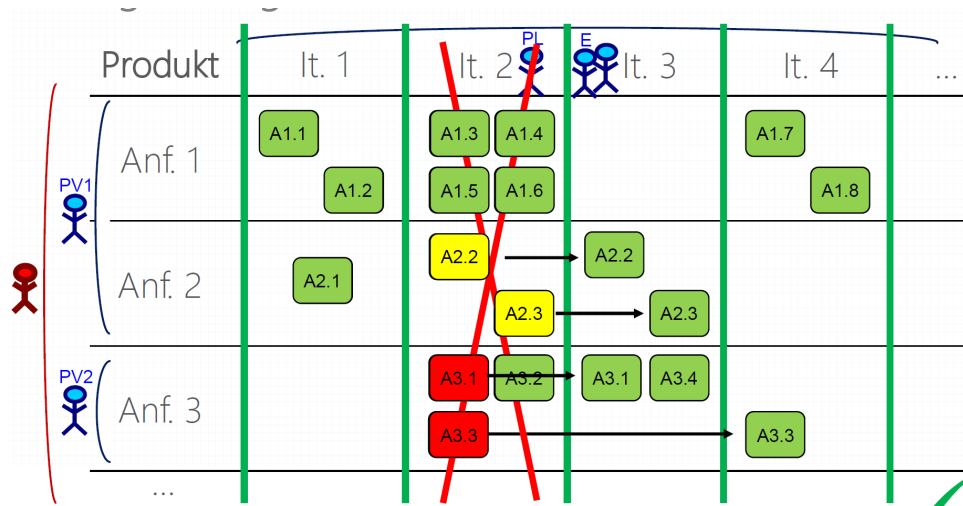
9.3 Aufgabenübersicht (Task Board)

Darstellung der **Wertschöpfungskette je Aufgabe** einer Iteration (eine gemeinsame Übersicht pro Team!)

Anf./Aufg.	SEL	DEV	TES	INT	REL
Anforderung 2					
Aufgabe 2.1					
Aufgabe 2.3					
Aufgabe 2.4					
Anforderung 3					
Aufgabe 3.3					
Aufgabe 3.4					
Anforderung 5					
Aufgabe 5.1					
Aufgabe 5.2					
...					

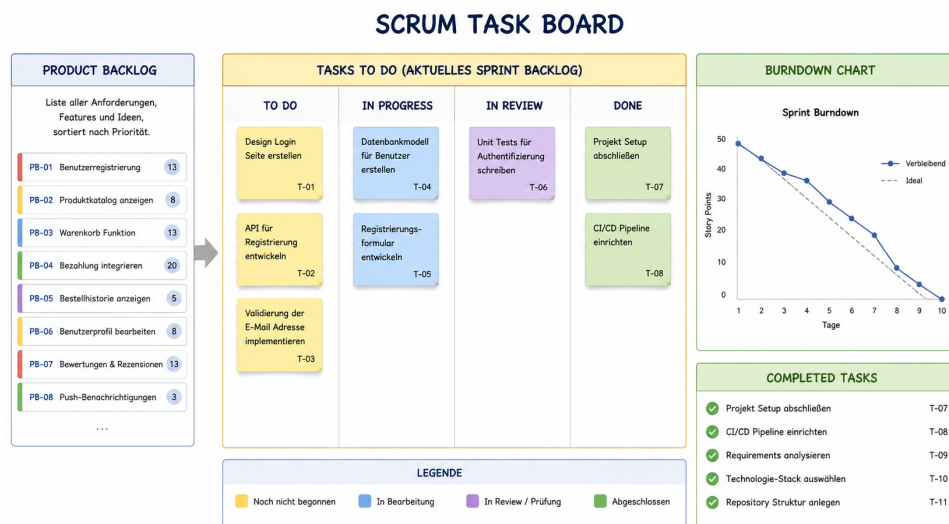
Festgelegte Regeln für den Spaltenübergang sind wichtig (-> **Definition of Done**)

9.4 Anforderungen/Aufgaben versus Iterationen



1. Plane für eine Iteration Aufgaben im geeigneten Umfang.
2. Schließe zumindest die Aufgaben einer Anforderung je Iteration ab.
3. Schließe nicht abgeschlossene Aufgaben in der nächsten Iteration ab.
4. Entscheide für jede abgebrochene Aufgabe, ob und in welcher Iteration sie fortgeführt wird.

9.5 Scrum Task Board



9.6 Kanban

Kanban: von japanischen Kan = Signal; Ban = Karte

- nach dem 2. Weltkrieg in Japan erfunden (T. Ohno, Toyota)
- Produktablaufsteuerung zur Optimierung des Lagerstands
- betont das Pull-Prinzip (Zulieferung je nach Verbrauch)

von D. Anderson **für IT-Projekte adaptiert** und um Konzepte der agilen Entwicklung erweitert (2007)

Beispiel: Kanban-Idee

Ein Softwareteam entwickelt eine To-do-App und verwendet ein Kanban-Board zur Aufgabenverwaltung.

- **To Do:** Neue Aufgaben wie „Login implementieren“ oder „Datenbank erstellen“ werden erfasst.
- **In Bearbeitung:** Ein Entwickler arbeitet aktuell an der Benutzerregistrierung.
- **Review/Test:** Die fertige Registrierung wird von einem Teammitglied geprüft und getestet.
- **Erledigt:** Nach erfolgreichem Test wird die Aufgabe in die Spalte *Erledigt* verschoben.

Das Team erkennt jederzeit den aktuellen Arbeitsfortschritt und kann Engpässe, z.B. zu viele Aufgaben in *In Bearbeitung*, sofort identifizieren.

9.6.1 Abgrenzung

Kein (agiles) Vorgehensmodell, sondern Entwicklungstechnik!

(aber beim agilen Vorgehen einsetzbar, z.B. in Scrum -> Scrumban)

- kein Requirements Management
- keine Teamorganisation, keine Rollen
- keine zeitlichen Vorgaben / Durchlaufzeiten -> keine Terminplanung
- **keine Iterationen!**, außer in Notfällen, aber inkrementell (falls sinnvoll)
- Releasezyklen frei wählbar
- Synchronisation der Aufgabenerledigung möglich
- auch für mehrere Projekte zur selben Zeit!

-> Gut einsetzbar für kleine, kurz getaktete, möglichst unabhängige Aufgaben (z.B. Wartungstätigkeiten)!

9.6.2 Empfehlungen

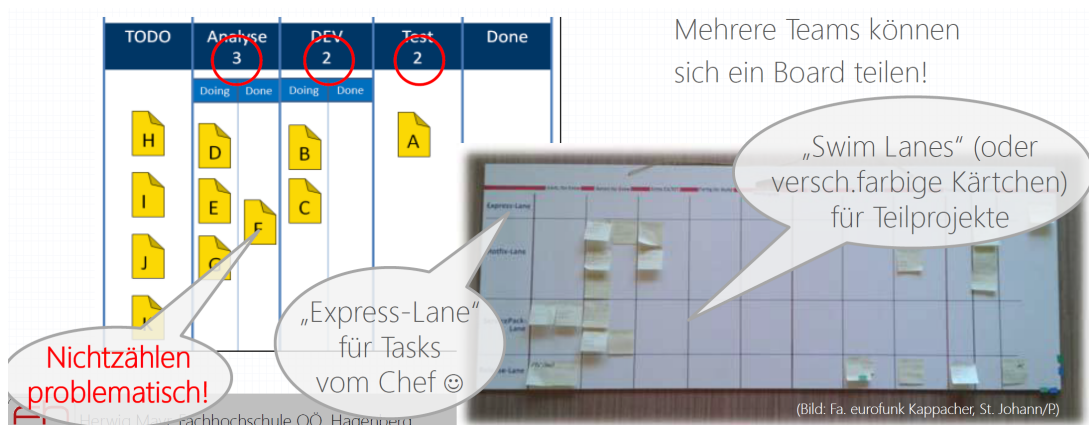
nach D. Anderson:

- **Visualisiere den Fluss der Arbeit.** (Kanban-Board)
- **Begrenze die Menge angefangener Arbeit.** (Ticket System, Pull-Prinzip)
- **Miss und steuere den Fluss.** (Cycle Time Analysis, Cost of Delay)
- **Mache die Regeln für den Prozess explizit.** (Definition of Done)
- **Verwende Modelle, um Chancen für kollaborative Verbesserungen zu erkennen.** (Kaizen)

9.6.3 Darstellung am Kanban-Board

Anzahl der **Elemente pro Spalte ist begrenzt** (außer Eingang + Ausgang)!

-> Zahl in Spaltenheader; Theory of Constraints



9.6.4 Kortschritt und Zielerreichung

Messung des Fortschritts:

- durch **Cycle Time Analysis**
- Zweck: gleichmäßige Arbeitsbelastung (bgl. 40 hour-week in XP, regular work load in Scrum)
- Notwendigkeit: gleich große Aufgaben (oft schwierig zu erreichen bzw. vorab schwer schätzbar)
- Nur dann ist die Zykluszeitmessung aussagekräftig! (Cycle Time beschreibt, wie lange Elemente x in Spalte y ist)

Messung der Zielerreichung:

- durch Anzahl der Elemente (**erledite Aufgaben**) in der rechten Spalte (Released, Done) - auf **Zuordnung zu Anforderungen** achten!

9.6.5 Hauptnutzen

Kanban Flow (am Kanban-Board): Grafische Visualisierung von **Fortschritt und Zielerreichung**

9.7 Tipps zum (agilen) Planen

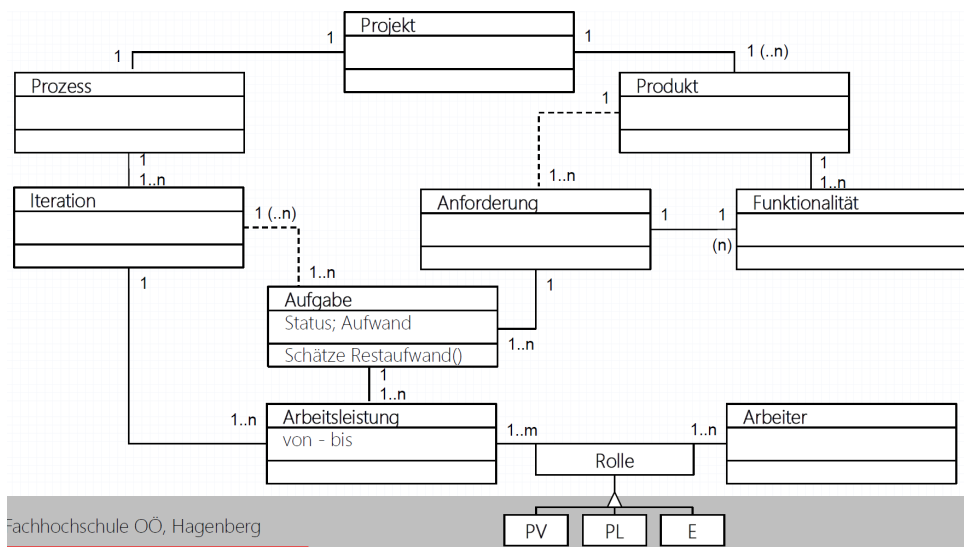
Was wird beim agilen Vorgehen geplant?

- **Entwicklung:** Struktur und Abläufe
- **Ergebnis:** Benutzung, Anwendungsfälle
- **Prozess:** (Um-)Planen aktueller/zukünftiger Iteration(en)

9.7.1 Statische Zusammenhänge

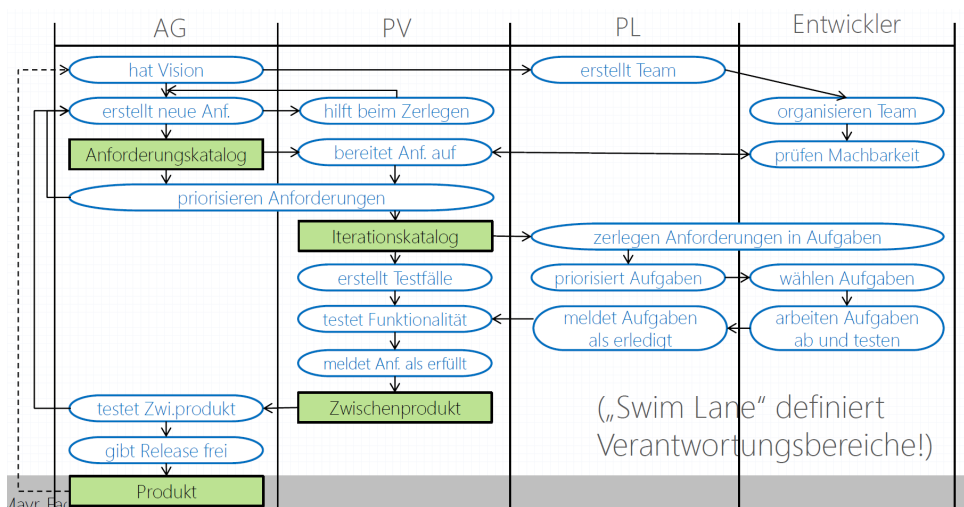
Struktur - gut darstellbar als (UML-)Klassendiagramm

Klassendiagramm am Beispiel Projekt:



9.7.2 Dynamische Zusammenhänge

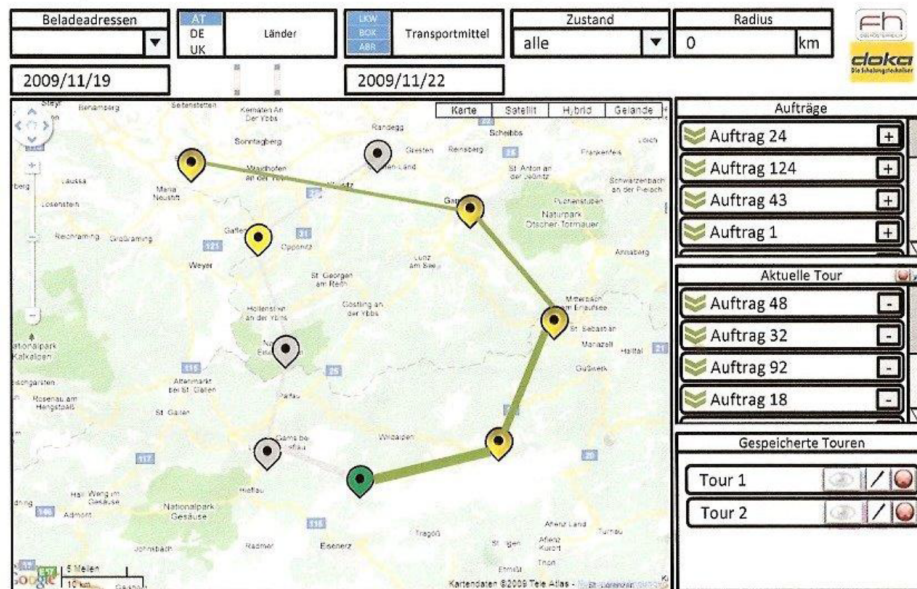
Abläufe gut darstellbar als (UML-)Schwimmbahndiagramm



9.7.3 Erarbeiten der Ablauflogik

Mittels **Benutzerschnittstellen-Prototyp** (am Anfang durchaus auf **Papier!**)

Mittels **Benutzerschnittstellen-Prototyp** (später als **Software**):



9.7.4 Erstellen typischer Anwendungsfälle

Diese geben die **Reihenfolge der Aufgabenrealisierung** vor!

9.7.5 Interne versus externe Anforderungen/Aufgaben

Festlegung der für den Auftraggeber sichtbaren Ergebnisse:

- **Externe Anforderungen/Aufgaben:** führen zu einem Ergebnis mit unmittelbarem Wert für den AG
- **Interne Anforderungen/Aufgaben:** dem AG nicht direkt verkaufbar, aus Projektsicht (AN) aber notwendig
- Einteilung über Projektverlauf einheitlich halten (**Checkliste**) Einheitlichkeit notwendig für Vergleiche zwischen Projekten (-> **Management-Overhead!**)

9.7.6 Sonderfall Forschungsaufgabe

Research Task oder **Spike**: ERgebnis für Projekt wichtig, aber Erreichungswahrscheinlichkeit und -grad unsicher

Vorgehen:

- Anlegen einer Aufgabe **ohne** Zeiteinschätzung
- Zuordnung von Mitarbeitern (Jeder Mitarbeiter darf **max eine** Forschungsaufgabe gleichzeitig bearbeiten!)
- besonderer Berücksichtigung in der Risikobehandlung
- laufendes Buchen des Aufwands auf die Aufgabe
- täglicher Bericht über den Status der Aufgabe durch Mitarbeiter
- jederzeit Abbrechen der Aufgabe durch den Produktverantwortlichen möglich (Plan B ist sinnvoll!)

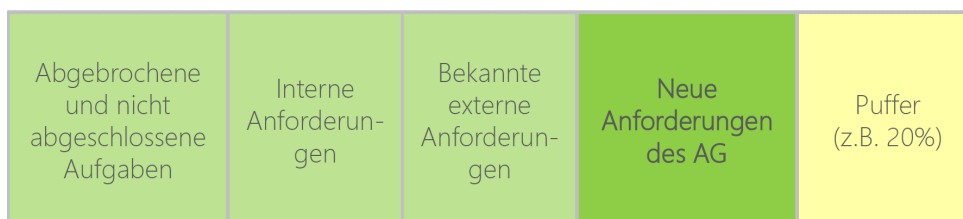
9.7.7 Umplanen der aktuellen Iteration

Gründe für kurzfristige Umplanungen:

- Mitarbeiter schließen Aufgaben früher als erwartet ab (priorisierte Liste weiterer Aufgaben vorhalten!)
- Aufgaben werden abgebrochen (Plan B soll vorhanden sein!)
- Aufgaben werden obsolet (vom AG nicht länger benötigte Funktionalität)
- Störgrößen beeinflussen den Projektverlauf (Krankheit, Marktveränderung, ...)

9.7.8 Vorplanen zukünftiger Iterationen

Teilbereiche:



9.8 Tipps zum (agilen) Durchführen

- **Arbeitszeit** erfassen
- **Restaufwand** schätzen
- **Aufgabenstatus** aktualisieren

9.8.1 Erfassen der Arbeitszeit

Prinzipien:

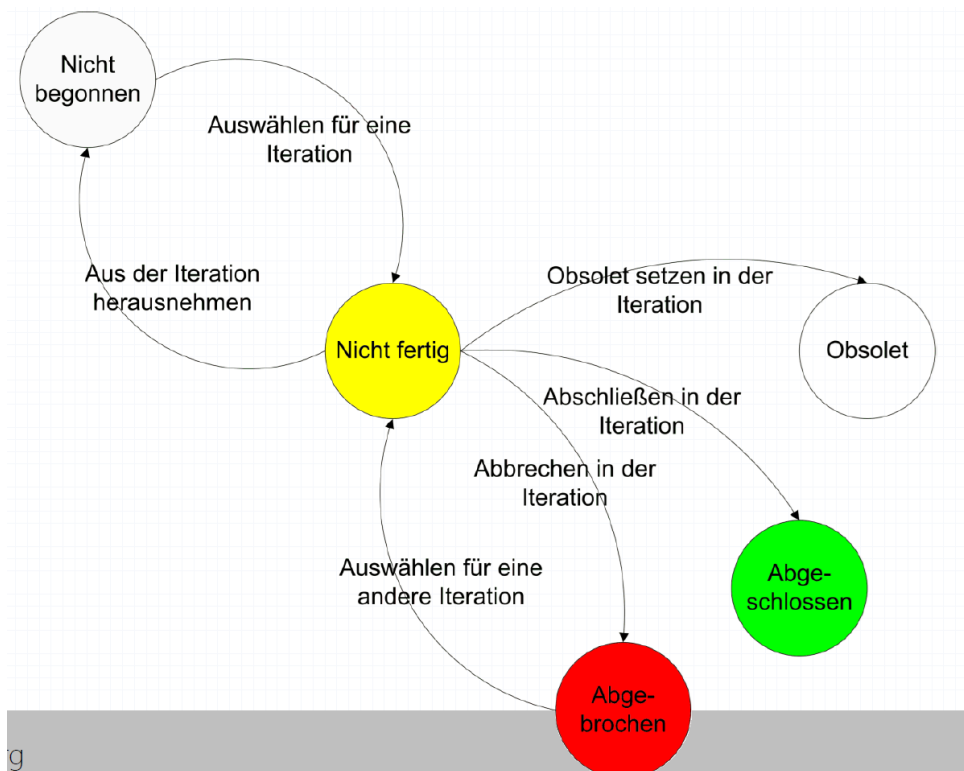
- Alle auf das Projekt zu buchende Arbeit wird erfasst (evtl. internes und externes Projektlogbuch).
- Jede Arbeit(seinheit) wird einer Aufgabe zugeordnet (ggf. **vorher** Aufgabe erstellen!).
- Jeder Mitarbeiter erfasst jeden Tag seine Arbeit.
- Eine sinnvolle kleinste Einheit ist zu definieren (z.B. 0.5 Stunden).

9.8.2 Schätzen des Restaufwands

Jeden Tag erneut; Wert kann auch erhöht werden!

9.8.3 Ändern des Zustands von Aufgaben (Status)

Möglichst mittels **Tracking-System** und **Tickets**:

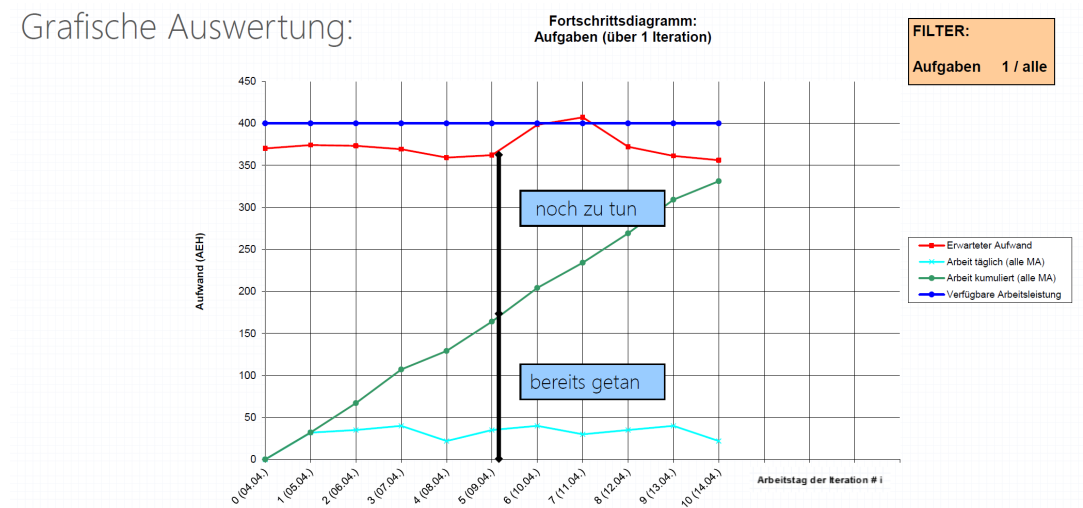


9.9 Tipps zum (agilen Überprüfen)

- **Fortschritt und Zielerreichungsgrad** festlegen
- **Management-Overhead** analysieren
- **Mitarbeiterbelastung & Schätzqualität** feststellen
- **Prozess** der Softwareentwicklung verbessern

9.9.1 Aufgaben - Fortschritt / Zielerreichung

Grafische Auswertung:



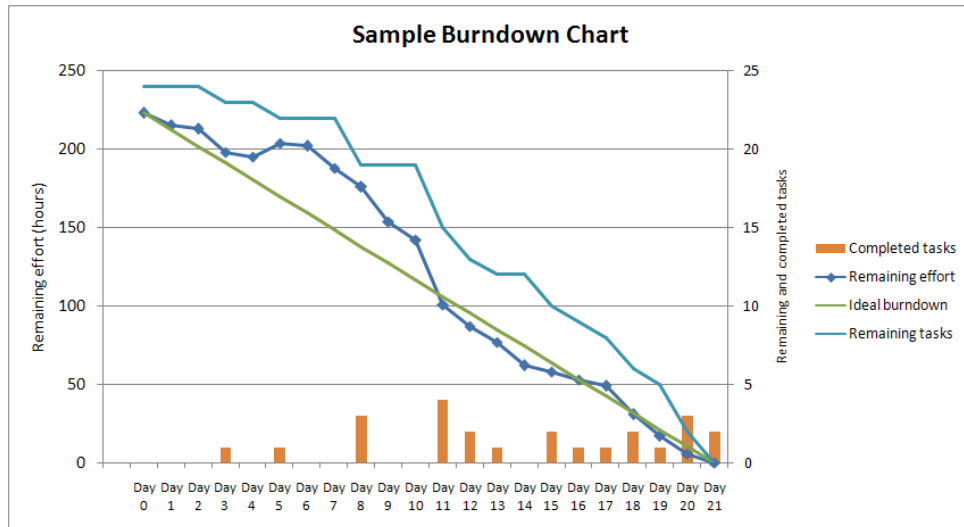
Gliederung nach Anforderungen für AG-Sicht:

(mind. 1 Anforderung pro Iteration erfüllt, Ampel-Metapher!)

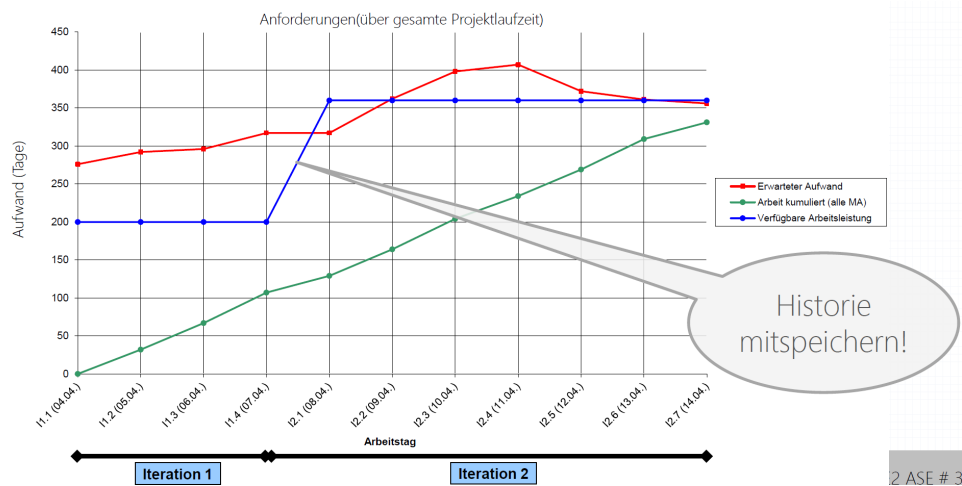
Projekt - Iterationen - 1. Iteration		
Status	Nummer	1
7 Erweiterung des Datenmodells 7.1 Vereinsnummer hinzufügen 7.2 Bezeichner für Ergebnisse in Datenmodell einfügen 7.3 Datenmodell diskutieren 1 Aspektbewertung am PDA 1.1 Kriterien- und Aspektbewertung mit Punktegenerator verbinden 1.2 Formular für die Aspekte erstellen 1.3 Aspektformulardaten speichern 1.4 Aspektformulardaten auslesen 4 Punktegenerator 4.1 Logik überlegen 4.2 Logik implementieren 2 Generierung eines Gesamtberichts 2.1 Testdaten erstellen 2.2 Gesamtbericht testen 2.3 Vorhandenen Gesamtbericht bearbeiten	Arbeitstage gesamt	7
	Beginn	28.04.2005
	Ende	13.05.2005
	Voraussichtlicher Gesamtaufwand	211
	- Bisheriger Aufwand	193
	Offener Restaufwand	18
	Verfügbare Arbeitszeit	168
	- Voraussichtlicher Gesamtaufwand	211
	Reserve	-43

9.9.2 Anforderungen - Fortschritt / Zielerreichung

Grafische Auswertung: **Sprint Burndown Chart** (1 Iteration; aus Scrum)



Grafische Auswertung: **Fortschrittsdiagramm** (mehrere Iterationen)



9.9.3 Management-Overhead

Ermittlung des **Management-Overhead** (= Verhältnis zwischen internen und externen **Aufwänden** im Projekt)

Zweck: Darstellung des Management-Overheads als

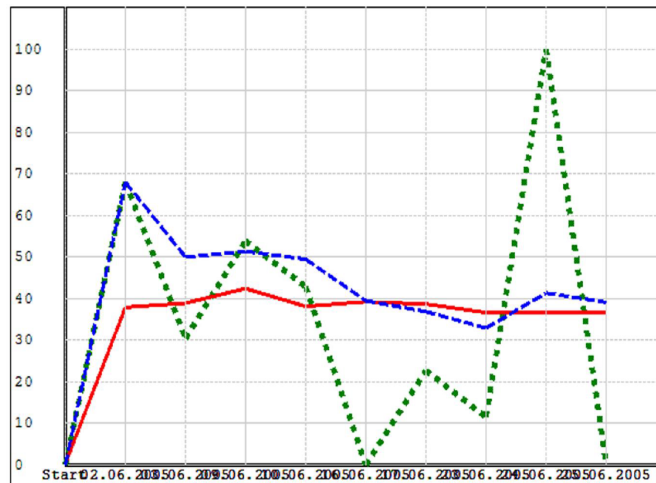
1. notwendig
2. begründbar
3. in der Höhe akzeptabel

Arten:

- **realer Management-Overhead** (Gegenwart)
- **tendenzieller Management-Overhead** (Vergangenheit)
- **erwarteter Management-Overhead** (Vergangenheit + Zukunft)

3. Iteration

- ☒ Realer Management-Overhead
- ☒ Tendenzieller Management-Overhead
- ☒ Erwarteter Management-Overhead



Vorteile eines explizit ausgewiesenen Management-Overhead:

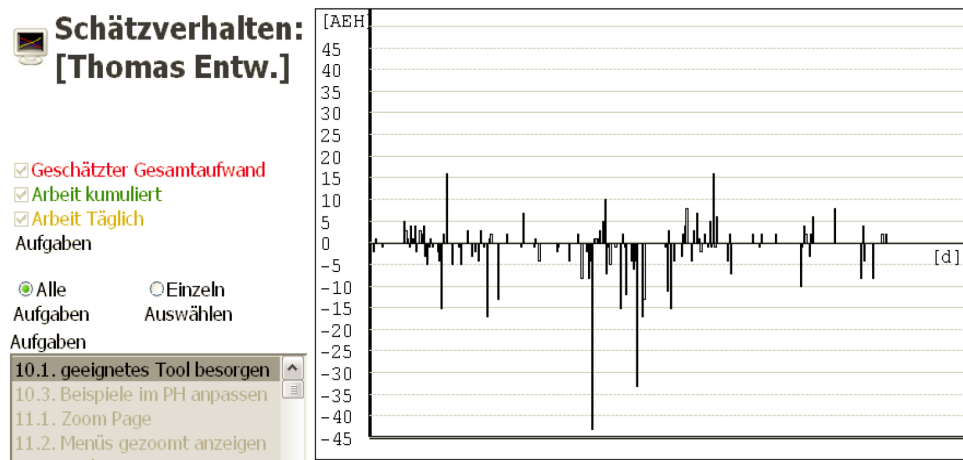
- leichteres Erkennen von overmanaged Projekten
- präziser ermittelbare Aufschläge
- genauere Kontrolle aus Managementsicht
- bessere Reflektion vergangener Projektphasen
- besserer Vergleich von projektphasen und Projekten

9.9.4 Auslastung der Mitarbeiter

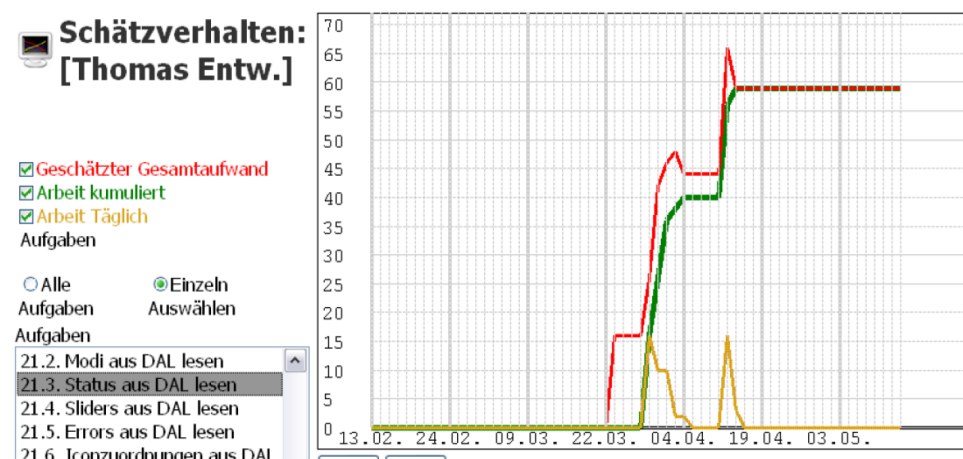
Sinnvoll inklusive Zuordnung zu den Aufgaben

9.9.5 Analyse des Schätzvermögens

Sinnvoll nur bei laufender Soll-Planung und Ist-Erfassung!



Detaillierte Analyse pro Aufgabe:



9.9.6 Potenzial der Prozessverbesserung

Rückbetrachtung (**Retrospektive**) nach jeder Iteration

z.B. in Scrum nach folgenden Regeln:

1. **Schaffe Sicherheit** (Wir suchen keine Fehler/Schuldige, sondern Verbesserungsmöglichkeiten)
2. **Sammle die Fakten** (Was war wichtig?)
3. **Finde funktionierende Prozesse** (Was hat gut funktioniert?)
4. **Finde nicht funktionierende Prozesse** (Was hat nicht gut funktioniert?)
5. **Finde die Kompetenz** (Wer kann was am verbessern?)
6. **Priorisiere die Verbesserungsmöglichkeiten** (drei wichtigste positive und drei wichtigste negative Erkenntnisse sichtbar platzieren)

10 Trends bei moderner (agiler) Softwareentwicklung

10.1 Beispiele agiler Vorgehensmodelle

10.1.1 Prozessmodell-Topologie

entsprechend verschiedener Gültigkeitsbereiche:

Modell	Gültigkeitsbereich
Unternehmensprozess-Modell	Unternehmenskultur (QM, Teambildung, Arbeitszeit, ...)
Projektprozess-Modell	gesamter Projektenentwicklungszyklus
Entwicklungsprozess-Modell	Designen-Implementieren-Testen-Zyklus (D-I-T)

Tabelle 4: Prozessmodell-Topologie

Jedes Modell mit breiterem Gültigkeitsbereich braucht ein geeignetes Modell mit engerem Gültigkeitsbereich!

10.1.2 Prozess-Skalierung

Prozessmodelle passen nie exakt für ein neues Projekt!

-> **Prozess-Skalierung** auf Grund von:

- Produktprofil
- Anwendungsdomäne
- Rahmenbedingungen

Prozess-Skalierung gibt es im agilen Bereich (z.B. Crystal) wie auch im nicht agilen Bereich (z.B. V-Modell-XT)

Nicht nur Prozess anpassen, sondern auch Werkzeuge!

Prozess ist auch **während** eines Projekts anpassbar.

10.1.3 Agile Vorgehensmodelle

Projektprozess-Modelle:

- **ADM (ASD):** Adaptives Vorgehensmodell für dynamische Projekte.
- **Crystal Family:** Methodenfamilie, angepasst an Teamgröße und Projektkritikalität.
- **Lean Development:** Fokus auf Verschwendungsreduzierung und kontinuierliche Verbesserung.
- **Scrum:** Iteratives Vorgehensmodell mit kurzen Entwicklungszyklen (Sprints).

Entwicklungsprozess-Modelle:

- **DSDM (RAD):** Schnelle Entwicklung durch Prototyping und enge Kundenbeteiligung.
- **FDD:** Entwicklung in kleinen, funktionsorientierten Arbeitspaketen.
- **Open Source Development:** Kollaborative Entwicklung durch eine offene Entwicklergemeinschaft.
- **XP (Extreme Programming):** Fokus auf Codequalität, Tests und enge Zusammenarbeit.

Scrum

- **Kategorie:** Projektprozess-Modell
- **Guru(s):** Ken Schwaber, Jeff Sutherland, Mike Beedle
- **Prinzipien:** aus der Verfahrenstechnik abgeleitet (Softwareentwicklung ist empirischer Prozess)
- **Praktiken:** Selbstorganisation, Anpassbarkeit (auch des Prozesses während des Ablaufs), 30-Tage-Sprints mit täglichen kurzen Treffen zur Abstimmung, nach 30 Tagen Abgleich mit (ev. geänderten) Anforderungen

10.2 Agile Vorgehensmodelle für größere Unternehmen

10.2.1 Beobachtungen zu agilen Vorgehesmodellen

- Agile Vorgehen werden **reifer**.
- Agile Vorgehen werden **für größere Unternehmen** angepasst.
- Agile Vorgehen werden immer **träger** (fetter) und dadurch weniger agil.

10.2.2 Berispiel: Scrum wird immer fetter

- The Return of the **Burndown Chart**
 - ursprünglich durch Scrum Board abgelöst
 - auch als **Burnup-Charts** (-> Fortschrittsdiagramm)
- Immer größere Diversifizierung der Rollen:
 - **Customer** vs **User**
 - **Manager**
- Ausweitung auf dislozierte Teams:
 - **Distributed Scrum** (z.B. Distributed Scrum Primer)

10.2.3 Agile Vorgehen werden breiter einsetzbar

Vorgehensmethoden

- Disciplined Agile [Delivery]
- Scaling Scrum:
 - Scrum of Scrums, Scrum@Scale
 - Spawned Scrum Teams
 - Large-Scale Scrum, Nexus
- Component Teams und Feature Teams (Chapter, Guilds, Squads, Tribes)
- Scaled Agile Framework

Praxisbeispiele

- BMW i-Sparte, Deutschland
- LEGO DIGITAL Solutions, Dänemark
- Intuit, USA